Python for Scientific Computing

2026-10-16

Table of contents

Python for Scientific Computing **10**
What Makes This Book Different? . . . 10
Book Organization . . . 11
How to Get the Most Out of This Book . . . 12

Getting Started **13**

Setting up Anaconda and Spyder **14**

Chapter 0: Setting up Anaconda and Spyder **15**
0.1 Downloading and Installing Anaconda . . . 15
What is Anaconda? . . . 15
Platform Instructions . . . 16
0.2 Anaconda Navigator vs Anaconda Prompt, and Launching Spyder . . . 17
0.3 Creating and Activating a Conda Environment . . . 17
Essential Conda Commands . . . 18
0.4 Checking Installed Versions . . . 18
0.5 Tour of the Spyder 6 IDE . . . 19
Key Panes: . . . 19
0.6 Running Code in Spyder: Three Ways . . . 19
Try it Yourself: Code Cells (`#%%`) . . . 20
0.7 Setting the Working Directory and Project Folders . . . 20
Best Practice for College Labs: . . . 21

Core Data Structures and Functions **22**

Lists **23**

Chapter 1: Lists **24**
1.1 Initializing Lists and Data Types . . . 24
Runnable Example . . . 24
1.2 1D and 2D Lists (Lists of Lists) . . . 25
Runnable Example . . . 25
1.3 Mutability: In-Place Changes, Aliasing vs Copying . . . 25
Runnable Example . . . 26
1.4 `len()` for Lists and Sublists . . . 26
Runnable Example . . . 26
1.5 Extracting Elements: Indexing and Slicing . . . 27
Runnable Example . . . 27

1.6 Iterating with a `for` Loop . . . 27
Runnable Example . . . 27
1.7 `range()`: Syntax, Usage, and Limitations . . . 28
Runnable Example . . . 28
1.8 List Comprehensions for 1D and 2D Lists . . . 28
Runnable Example . . . 29
1.9 Creating a List of Floats Using `range()` . . . 29
Runnable Example . . . 29
1.10 Converting to a List with `list()` . . . 29
Exercises and Mini-Lab . . . 30
Exercise 1: Row and Column Sums . . . 30
Mini-Lab: Multiplication Table Generator . . . 31

Functions **32**

Chapter 2: Functions **33**
2.1 Why Functions? . . . 33
2.2 Defining a Function and `return` . . . 33
Runnable Example . . . 34
2.3 Formal vs Actual Parameters . . . 34
2.4 Positional, Keyword Arguments, and Defaults . . . 34
Runnable Example . . . 35
2.5 The `*` Operator: `*args` and Unpacking . . . 35
Runnable Example . . . 35
2.6 Lambda Functions (Anonymous Functions) . . . 36
Runnable Example . . . 36
2.7 Variable Scope and the Stack Frame . . . 36
Runnable Example . . . 37
2.8 Docstrings and Spyder's Help Pane . . . 37
2.9 The `global` Keyword . . . 38
Runnable Example . . . 38
2.10 Recursion: Base Case and Call Stack . . . 38
Runnable Example: Factorial . . . 39
2.11 Functions as First-Class Objects . . . 39
Runnable Example . . . 39
Exercises and Mini-Lab . . . 40
Exercise 1: Fibonacci Sequence (Iterative vs Recursive) . . . 40
Mini-Lab: Mathematical Function Table Helper . . . 40

Modules **41**

Chapter 3: Modules **42**
3.1 Importing Modules: The Three Syntax Forms . . . 42
Runnable Example . . . 43
3.2 Importing Part of a Module . . . 43
3.3 Module-Name Rules and Abstraction . . . 43
3.4 The Built-in `math` Module: Common Functions . . . 44
3.5 Module Structure: `if __name__ == "__main__"` . . . 44

3.6 Creating Your Own User-Defined Module . . . 44
Step 1: Create `physics_tools.py` . . . 45
Step 2: Import It in a Separate Script `lab_experiment.py` . . . 45
3.7 The Module Search Path & Spyder Import Errors . . . 45
Exercises and Mini-Lab . . . 46
Exercise: Personal Statistics Module (`my_stats.py`) . . . 46

Milestone Project 1: Utility Toolkit **47**

Milestone Project 1: Scientific Utility Toolkit **48**
1. Module Source Code: `sci_toolkit.py` . . . 48
2. Test Driver Script: `run_toolkit_tests.py` . . . 51

Numerical Computing with NumPy **54**

NumPy Arrays **55**

Chapter 4: NumPy Arrays **56**
4.1 Importing NumPy . . . 56
4.2 The `ndarray` as the Core Structure . . . 56
Runnable Example . . . 57
4.3 Homogeneity and Multidimensionality . . . 57
Upcasting in Action . . . 58
4.4 Creating Arrays and Specifying `dtype` . . . 58
Runnable Example . . . 58
4.5 Inspecting Arrays: `shape`, `ndim`, and `size` . . . 58
4.6 Initializing with `np.zeros()` and `np.ones()` . . . 59
4.7 `np.arange()` vs `np.linspace()` (When to Use Which) . . . 59
Runnable Example . . . 60
4.8 Reshaping Arrays with `np.reshape()` . . . 60
Runnable Example . . . 60
4.9 Indexing and Slicing in 1D and 2D . . . 61
Runnable Example . . . 61
4.10 Viewing Arrays in Spyder's Variable Explorer . . . 61

Array Methods, Operations and Sorting **62**

Chapter 5: Array Methods, Operations, and Sorting **63**
5.1 Basic Methods: `sum()`, `max()`, `min()`, and `T` . . . 63
Runnable Example . . . 63
Transposing a Matrix . . . 64
5.2 Operating Over Axes: Rows vs Columns . . . 64
Runnable Example . . . 64
5.3 Sorting Arrays . . . 65
Runnable Example . . . 65
5.4 Elementwise Mathematical Functions . . . 65
Runnable Example . . . 65
5.5 Vectorization vs Loops: Speed Benchmark . . . 66

5.6 Sequencing Arrays: `np.vstack()` and `np.hstack()` . . . 66
Runnable Example . . . 67

Random Numbers in NumPy **68**

Chapter 6: Random Numbers in NumPy **69**
6.1 Creating a Modern Generator: `default_rng()` . . . 69
6.2 Generating Random Floats and Integers . . . 69
Runnable Example . . . 69
6.3 Seeding for Reproducibility . . . 70
6.4 Scientific Simulations . . . 70
1. Coin Toss Experiment (Law of Large Numbers) . . . 70
2. The 1D Random Walk (Brownian Motion Model) . . . 71

Linear Algebra with numpy.linalg **72**

Chapter 7: Linear Algebra with `numpy.linalg` **73**
7.1 Determinant and Matrix Inverse . . . 73
Runnable Example . . . 73
7.2 Eigenvalues and Eigenvectors . . . 74
Runnable Example . . . 74
7.3 Solving Systems of Linear Equations . . . 74
Runnable Example . . . 75
7.4 Types of Products: Dot, Matmul, and `@` . . . 75
Runnable Example . . . 76
Exercises and Mini-Lab: Electrical Circuit Analysis . . . 76
Problem: Kirchhoff's Mesh Current Method . . . 76

Milestone Project 2: Numerical Experiments **77**

Milestone Project 2: Numerical Experiments **78**
Complete Project Code: `numerical_experiments.py` . . . 78

Visualization with Matplotlib **82**

Basic Plotting **83**

Chapter 8: Basic Plotting **84**
8.1 Importing Matplotlib . . . 84
8.2 Line Plots, Scatter Plots, and Filled Curves . . . 85
Runnable Example . . . 85
8.3 Styling: Lines, Markers, Colors, and Sizes . . . 86
Quick Style Cheat Sheet: . . . 86
8.4 Titles, Labels, and Font Sizes . . . 86
8.5 Axis Limits and Ticks . . . 86
8.6 Legends . . . 87
8.7 Spyder's Plots Pane . . . 87
8.8 Saving Publication-Ready Figures . . . 87

Figures, Axes and Subplots **88**

Chapter 9: Figures, Axes, and Subplots **89**
9.1 The Object-Oriented Approach: `fig, ax` . . . 89
Runnable Example . . . 90
9.2 Creating Subplot Grids: `plt.subplots(rows, cols)` . . . 90
9.3 Plotting on Specific Axes . . . 91
Runnable Example: A 2×2 Scientific Dashboard . . . 91
9.4 `plt.subplot()` for Individual Axes . . . 92
Exercises and Mini-Lab: Function Family Exploration . . . 92

Milestone Project 3: Plotting Dashboard **94**

Milestone Project 3: Scientific Plotting Dashboard **95**
Complete Project Code: `vibration_dashboard.py` . . . 95

Files and Text Formatting **98**

File Handling and I/O **99**

Chapter 10: File Handling and I/O **100**
10.1 File Objects and `open()` . . . 100
10.2 File Object Attributes . . . 100
10.3 File Access Modes . . . 101
10.4 Reading Data: `read()`, `readline()`, and `readlines()` . . . 101
Runnable Example . . . 101
10.5 Writing Data: `write()` and `writelines()` . . . 102
Runnable Example . . . 102
10.6 The `with` Statement (Context Manager) . . . 102
10.7 Reading and Writing Numerical Data with NumPy . . . 103
Runnable Example . . . 103

String Formatting **104**

Chapter 11: String Formatting **105**
11.1 Modern f-strings . . . 105
Runnable Example . . . 105
11.2 Format Codes, Precision, Width, and Alignment . . . 106
1. Type Codes: . . . 106
2. Alignment Options: . . . 106
Runnable Example . . . 106
11.3 Legacy `%` Formatting (C-Style) . . . 107
11.4 The `str.format()` Method . . . 107
11.5 Comparing the Three Approaches . . . 107
Exercises and Mini-Lab: Formatted Laboratory Data Table . . . 107

Milestone Project 4: Data Logger and Report Writer **109**

Milestone Project 4: Data Logger and Report Writer **110**
Complete Project Code: `thermal_logger.py` . . . 110

Reliable Code 114

Testing and Debugging 115

Chapter 12: Testing and Debugging **116**
12.1 Black-Box vs Glass-Box Testing . . . 116
12.2 Designing Test Cases: Typical, Edge, and Boundary Values . . . 117
Runnable Example: Testing a Velocity Calculator . . . 117
12.3 Types of Bugs . . . 118
12.4 Hypothesis-and-Confirmation Debugging . . . 118
12.5 IDE-Based Debugging in Spyder . . . 119
Debugging Steps: . . . 119
12.6 Debugging a Recursive Function Step by Step . . . 119

Exceptions 121

Chapter 13: Exceptions **122**
13.1 Common Exceptions in Scientific Computing . . . 122
13.2 The `try`–`except` Block . . . 123
13.3 Handling Specific Exceptions & Multiple Clauses . . . 123
Runnable Example . . . 123
13.4 `else` and `finally` Clauses . . . 124
Runnable Example . . . 124
13.5 Raising Exceptions with `raise` . . . 125
13.6 Passing Exceptions Up the Call Stack . . . 125
Exercises and Mini-Lab: Safe Experimental Calculator . . . 126

Capstone Project: Robust Data Analysis Pipeline 127

Capstone Project: Robust Data Analysis Pipeline **128**
1. Pipeline Architecture . . . 128
2. Complete Pipeline Implementation: `pipeline_capstone.py` . . . 129

Appendices 135

Appendix A: Spyder Shortcuts Cheat Sheet 136

Appendix A: Spyder Keyboard Shortcuts Cheat Sheet 137

Appendix B: NumPy Quick Reference 139

Appendix B: NumPy Quick Reference **140**
1. Array Creation . . . 140
2. Array Inspection and Reshaping . . . 140
3. Mathematical and Statistical Reductions . . . 140

4. Stacking and Joining . . . 141
5. Linear Algebra (`np.linalg`) . . . 141

Appendix C: Matplotlib Properties Reference **142**

Appendix C: Matplotlib Property Reference **143**
1. Line Styles (`linestyle` or `ls`) . . . 143
2. Marker Styles (`marker`) . . . 143
3. Common Color Codes (`color` or `c`) . . . 143
4. Key Plot Method Arguments . . . 144

Appendix D: f-string Format Code Reference **145**

Appendix D: f-string Format Code Reference **146**
1. Type Conversion Specifiers . . . 146
2. Width, Alignment, and Padding . . . 146
3. Useful Debugging Shorthand (Python 3.8+) . . . 146

Appendix E: Common Error Messages and Fixes **148**

Appendix E: Common Error Messages and Fixes **149**
1. `IndexError: list index out of range` . . . 149
2. `TypeError: can only concatenate str (not "float") to str` . . . 149
3. `ValueError: could not convert string to float: '...'` . . . 149
4. `NameError: name 'np' is not defined` . . . 149
5. `IndentationError: expected an indented block` . . . 149
6. `ValueError: operands could not be broadcast together with shapes (4,) (3,)` 150
7. `ModuleNotFoundError: No module named '...'` . . . 150

Appendix F: Managing Packages with Conda and Pip **151**

Appendix F: Managing Packages with Conda and Pip **152**
1. Conda vs Pip . . . 152
2. Essential Conda Commands . . . 152
3. Environment Management Workflow . . . 153

Appendix G: Suggested Reading **154**

Appendix G: Suggested Reading **155**
1. Introductory Programming . . . 155
2. Scientific Python Ecosystem . . . 155

Appendix H: Solutions to Selected Exercises **156**

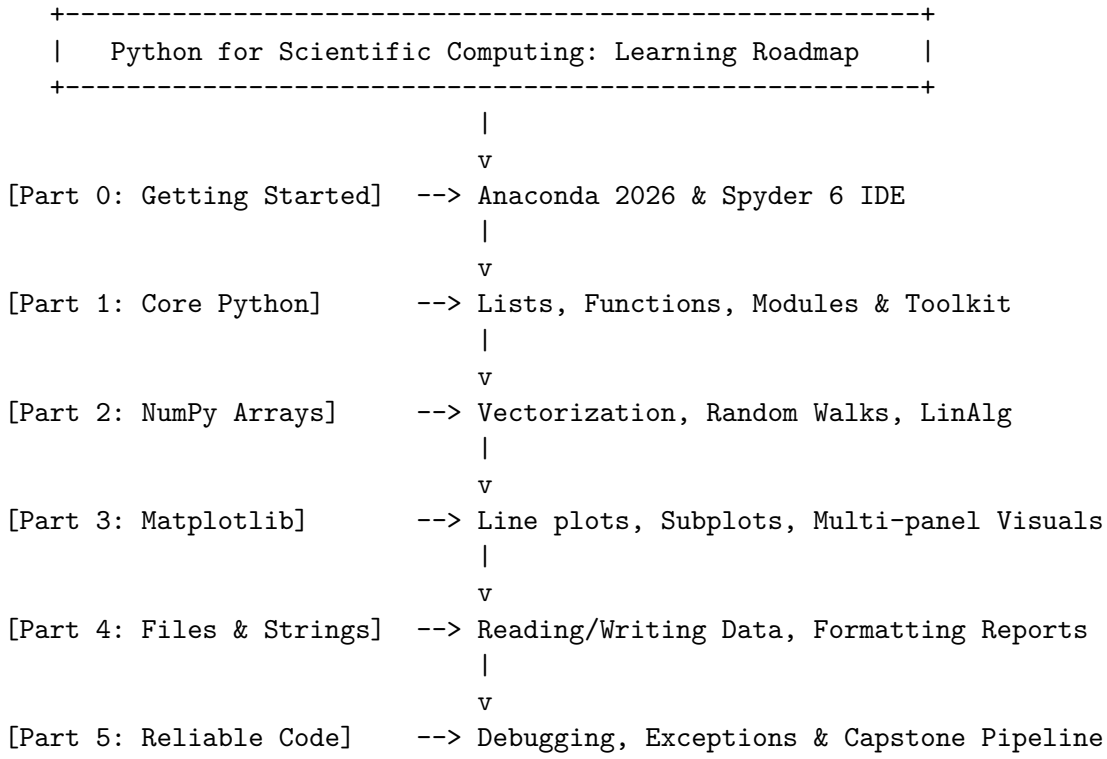
Appendix H: Solutions to Selected Exercises **157**
Chapter 1: Lists . . . 157
Exercise 1: Row and Column Sums . . . 157
Chapter 2: Functions . . . 158
Exercise 1: Recursive vs Iterative Fibonacci . . . 158

Chapter 7: Linear Algebra 158
Circuit Mesh Analysis 158
Chapter 13: Exceptions 159
Safe Calculator 159

Python for Scientific Computing

Welcome to **Python for Scientific Computing**! This book is written especially for college and university students who are stepping into the world of programming, data analysis, and scientific computing. Whether you are studying engineering, physics, chemistry, biology, mathematics, computer science, or economics, Python has become the lingua franca of technical problem solving.

You do **not** need prior programming experience to read this book. We start right at ground zero—installing your tools and writing your first lines of code—and gently guide you through structured problem-solving, numerical computation with **NumPy**, data visualization with **Matplotlib**, file processing, and robust debugging.



What Makes This Book Different?

- **Zero Assumed Knowledge:** Every programming concept is introduced with everyday analogies and plain-English explanations.

- **Hands-on & Runnable:** Self-contained, runnable code snippets you can copy, paste into the Spyder editor, and run with a single keypress (`F5` or `F9`).
- **Visual Learning:** Clear ASCII diagrams, memory box sketches, and data tables illustrate how lists, array axes, slices, and stack frames work inside the computer.
- **Built Around Spyder & Anaconda:** Written specifically around the Anaconda Distribution (2026 release) and Spyder 6.x, highlighting interactive features like the **Variable Explorer**, **Plots Pane**, and **Debugger Pane**.
- **Practical Milestone Projects:** Four milestone projects and a comprehensive Capstone Project tie everything together into real-world tools you can use in lab courses and research projects.

Book Organization

The book is structured into 6 parts and a comprehensive set of appendices:

1. **Part 0: Getting Started**
    - Setting up Anaconda and the Spyder 6 IDE across Windows, macOS, and Linux.
    - Mastering the Spyder interface: Editor, IPython Console, Variable Explorer, and Plots pane.
    - Managing project folders and running code cells (`#%%`).
2. **Part 1: Core Data Structures and Functions**
    - **Chapter 1: Lists** — 1D and 2D lists, mutability, indexing, slicing, loops, and list comprehensions.
    - **Chapter 2: Functions** — Defining reusable logic, parameters, return values, variable scope, docstrings, and recursion.
    - **Chapter 3: Modules** — Importing libraries, creating custom `.py` files, and understanding `if __name__ == "__main__"`.
    - **Milestone Project 1: Utility Toolkit** — Build a reusable math and statistics utility library.
3. **Part 2: Numerical Computing with NumPy**
    - **Chapter 4: NumPy Arrays** — The `ndarray`, multidimensionality, array creation (`zeros`, `ones`, `linspace`, `arange`), and slicing.
    - **Chapter 5: Array Methods, Operations and Sorting** — Vectorization, axis-based calculations, mathematical functions, and array stacking.
    - **Chapter 6: Random Numbers in NumPy** — Modern generators (`default_rng`), distributions, seeds, and simulations (coin tosses, random walks).
    - **Chapter 7: Linear Algebra with `numpy.linalg`** — Determinants, matrix inverses, eigenvalues, dot products, and solving systems of linear equations.
    - **Milestone Project 2: Numerical Experiments** — Simulating particle diffusion and solving circuit networks.
4. **Part 3: Visualization with Matplotlib**

- **Chapter 8: Basic Plotting** — Line charts, scatter plots, shaded fills, labels, legends, line styles, and saving publication-ready figures.
- **Chapter 9: Figures, Axes and Subplots** — The object-oriented `fig, ax` approach, multi-panel grids, and composite dashboards.
- **Milestone Project 3: Plotting Dashboard** — Designing a multi-panel visual telemetry dashboard for experimental data.

5. **Part 4: Files and Text Formatting**
   - **Chapter 10: File Handling and I/O** — Opening, reading (`read`, `readline`, `readlines`), writing, and safe resource cleanup with `with`.
   - **Chapter 11: String Formatting** — Modern f-strings, format specifiers (`f`, `e`, `d`), column alignment, and legacy formatting comparison.
   - **Milestone Project 4: Data Logger and Report Writer** — Building an automated data acquisition logger and formatted report generator.
6. **Part 5: Reliable Code**
   - **Chapter 12: Testing and Debugging** — Black-box vs glass-box testing, hypothesis debugging, and using Spyder's interactive visual debugger.
   - **Chapter 13: Exceptions** — Handling errors gracefully with `try…except…finally` and raising custom validation errors.
   - **Capstone Project: Robust Data Analysis Pipeline** — An end-to-end scientific pipeline that reads raw laboratory data, filters noise, calculates statistics with NumPy, plots figures with Matplotlib, and exports a formatted executive summary.
7. **Appendices**
   - **A.** Spyder keyboard shortcuts cheat sheet
   - **B.** NumPy function quick reference
   - **C.** Matplotlib styling and property cheat sheet
   - **D.** f-string formatting syntax reference
   - **E.** Common error messages and their solutions
   - **F.** Package management with Conda and Pip
   - **G.** Suggested reading and next steps
   - **H.** Complete solutions to selected chapter exercises

How to Get the Most Out of This Book

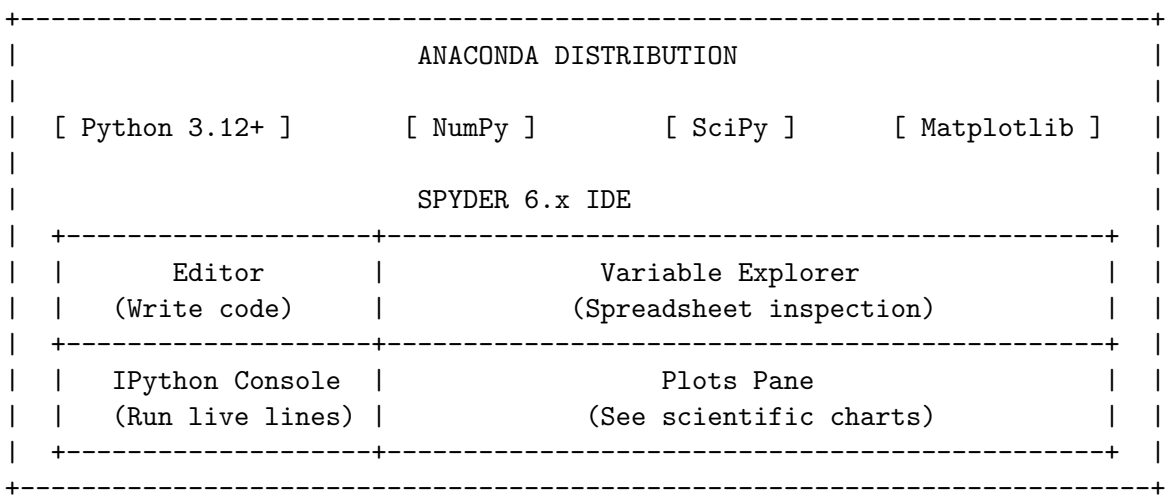
1. **Type the code yourself:** Avoid just reading passively. Open Spyder, type out the snippets in the editor, and run them using `F5` or `F9`.
2. **Experiment in the IPython console:** Whenever you wonder *"What happens if I change this number?"*, try it immediately in Spyder's interactive console.
3. **Inspect variables:** Keep an eye on Spyder's **Variable Explorer**. Double-click on lists and NumPy arrays to see their dimensions and contents visually in a spreadsheet-like grid.
4. **Solve the chapter exercises:** Each chapter concludes with practice exercises and a mini-lab designed to solidify your problem-solving intuition.

Getting Started

Setting up Anaconda and Spyder

Chapter 0: Setting up Anaconda and Spyder

Welcome to the start of your journey in scientific computing! Before we write any mathematical equations or simulate experiments in Python, we need a reliable workspace. In this chapter, we install the **Anaconda Distribution** and learn our way around **Spyder**, the scientific development environment bundled with Anaconda.



0.1 Downloading and Installing Anaconda

What is Anaconda?

Python by itself is just an execution engine. In scientific computing, you also need hundreds of specialized packages for matrix mathematics, statistics, graph plotting, and numerical simulation.

Installing them individually can lead to complex dependency conflicts. **Anaconda Distribution** solves this: it is a free, all-in-one package manager and software bundle containing Python, all major scientific libraries (NumPy, SciPy, Matplotlib, pandas), and IDEs like Spyder.

Platform Instructions

Windows

1. Visit https://www.anaconda.com/download and download the **Windows 64-Bit Graphical Installer**.
2. Run the downloaded `.exe` installer.
3. Click **Next** -> **I Agree**.
4. Choose **Just Me (recommended)**.
5. Choose the default destination folder (e.g., `C:\Users\<YourUsername>\anaconda3`).
6. In the *Advanced Options* screen:
   - Check **"Register Anaconda3 as my default Python"**.
   - You can leave *"Add Anaconda3 to my PATH environment variable"* unchecked (Anaconda provides its own dedicated command prompt).
7. Click **Install**. Once finished, click **Finish**.

macOS

1. Download the **macOS Graphical Installer** (choose *Apple Silicon* or *Intel*, matching your Mac).
2. Double-click the `.pkg` file and follow the on-screen prompts.
3. Accept the default location in your home directory.
4. Complete the installation.

Linux

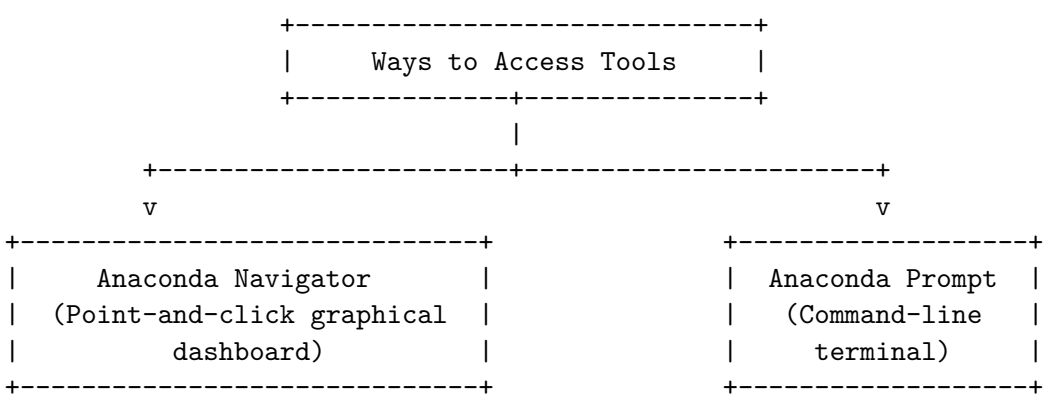
1. Download the installer script (`Anaconda3-xxxx.xx-Linux-x86_64.sh`).
2. Open your terminal, navigate to your downloads directory, and run:

```
bash Anaconda3-2026.xx-Linux-x86_64.sh
```

3. Press `Enter` to scroll through the license terms, type `yes` to accept, and press `Enter` to accept the default installation directory.
4. When prompted: `Do you wish to initialize Anaconda3 by running conda init?`, type `yes`.

0.2 Anaconda Navigator vs Anaconda Prompt, and Launching Spyder

Anaconda provides two ways to launch tools and manage packages:

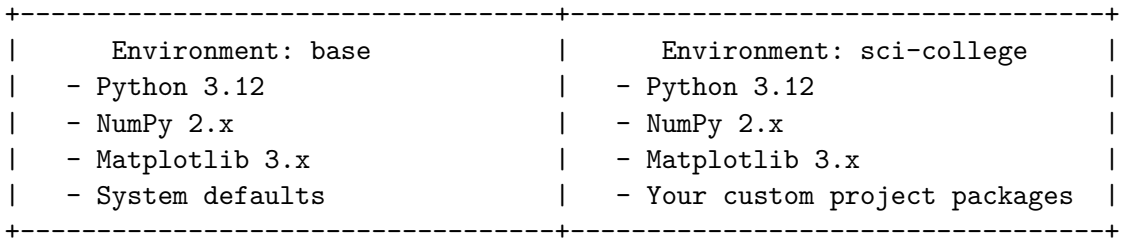


1. **Anaconda Navigator (GUI):** A visual desktop program.
   - Open your Start Menu (Windows), Applications folder (macOS), or run `anaconda-navigator` in Linux.
   - You will see cards for Jupyter Notebook, Spyder, and other tools.
   - Click the **Launch** button under the **Spyder** tile.
2. **Anaconda Prompt / Terminal (CLI):**
   - On Windows, search for **Anaconda Prompt** in the Start Menu.
   - On macOS/Linux, open your standard **Terminal**.
   - Simply type `spyder` and press `Enter`:

     ```
     spyder
     ```

0.3 Creating and Activating a Conda Environment

An **environment** is an isolated sandbox. Imagine having one project that requires a specific version of a library and another project that needs a newer version. Environments prevent projects from interfering with one another.



Essential Conda Commands

Let us create an environment named `college_env` for this book:

```
# 1. Create a new environment with Python and scientific packages
conda create -n college_env -c conda-forge \
    python=3.12 numpy matplotlib spyder -y

# 2. Activate the environment
conda activate college_env

# 3. Verify which environment is active (indicated by an asterisk *)
conda env list

# 4. To deactivate when you are done for the day:
conda deactivate
```

Once activated, your terminal prompt changes from `(base)` to `(college_env)`. You can now type `spyder` to launch Spyder configured with this environment.

0.4 Checking Installed Versions

Let us verify that Python, NumPy, and Matplotlib are properly installed. Open Spyder, look at the **IPython Console** in the bottom-right panel, type each of the following lines, and press `Enter`:

```
import sys
import numpy as np
import matplotlib

print("Python version:    ", sys.version.split()[0])
print("NumPy version:     ", np.__version__)
print("Matplotlib version:", matplotlib.__version__)
```

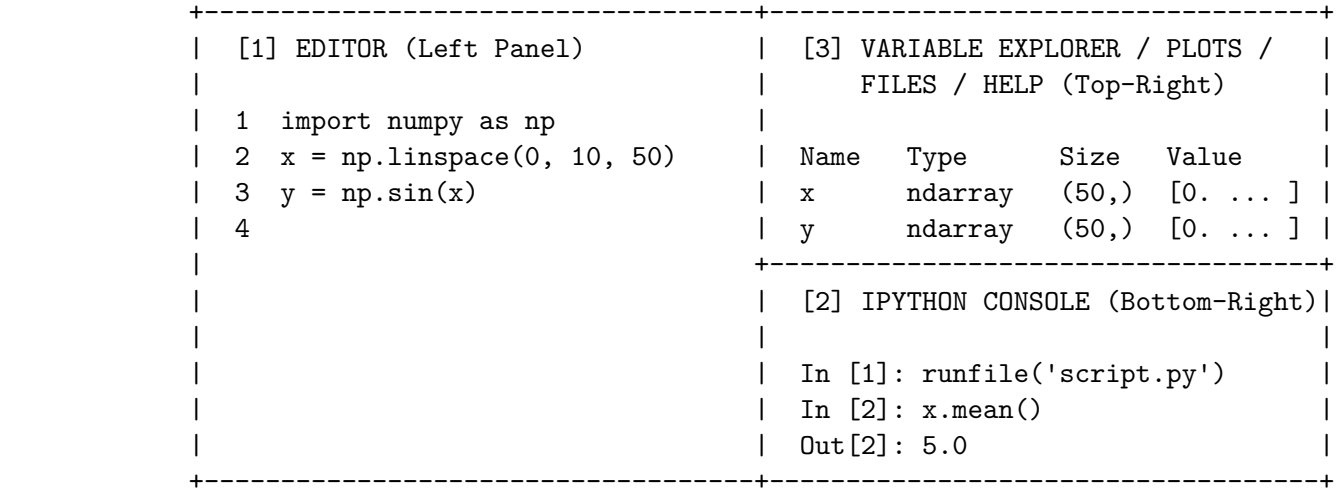
Expected output:

```
Python version:     3.12.x
NumPy version:      2.x.x
Matplotlib version: 3.x.x
```

If these three lines execute without error and display your version numbers, your scientific computing installation is complete and ready.

0.5 Tour of the Spyder 6 IDE

Spyder (Scientific Python Development Environment) is tailored specifically for scientists and engineers. Unlike generic text editors, Spyder includes built-in tools for visualizing data structures and plots.



Key Panes:

1. **Editor (Left):** Where you write, save, and edit Python files (`.py`). Features line numbers, syntax highlighting, and auto-indentation.
2. **IPython Console (Bottom-Right):** An interactive prompt where you can execute code immediately line-by-line, test ideas, and see immediate outputs.
3. **Variable Explorer (Top-Right):** One of Spyder's greatest superpowers! Every variable created in your code appears in a table showing its name, data type, size, and value. Double-clicking any list or array opens a graphical viewer.
4. **Plots Pane (Top-Right):** Whenever you generate a graph with Matplotlib, the image is captured here as a thumbnail gallery so you never lose previous figures.
5. **Files Pane (Top-Right):** A built-in file explorer to browse, open, and organize files in your current working folder.
6. **Help Pane (Top-Right):** Type any function name or press `Ctrl + I` (`Cmd + I` on Mac) while your cursor is on a function in the Editor to view its official documentation.
7. **Debugger Pane:** Allows stepping through code line-by-line during troubleshooting.

0.6 Running Code in Spyder: Three Ways

Spyder gives you three distinct ways to execute code, each suited for different tasks:

Method	Shortcut	Best Used For
1. Run File	F5	Running the complete script from start to finish.
2. Run Cell	Ctrl+Enter (Win/Lin) Cmd+Enter (Mac)	Running a dedicated block marked with #%% without rerunning all.
3. Run Selected Code	F9	Running only the highlighted lines of code.

Try it Yourself: Code Cells (#%%)

In Spyder, typing **#%%** at the beginning of a line splits your script into a separate, runnable **cell**. Copy this code into the Spyder Editor:

```
#%% Cell 1: Define parameters
initial_velocity = 25.0   # meters per second
gravity = 9.81            # m/s^2
time = 2.0                # seconds

#%% Cell 2: Calculate height of a projectile
# Formula: h = v0*t - 0.5*g*t^2
height = initial_velocity * time - 0.5 * gravity * (time ** 2)
print("Projectile height at t =", time, "seconds is", height, "meters")
```

1. Click anywhere inside **Cell 1** and press `Shift + Enter` (runs the cell and moves to the next).
2. Look at the **Variable Explorer**—you will see `initial_velocity`, `gravity`, and `time` appear instantly!
3. Press `Ctrl + Enter` inside **Cell 2**. The console prints:

   ```
   Projectile height at t = 2.0 seconds is 30.38 meters
   ```

4. Now `height` also appears in the Variable Explorer.

0.7 Setting the Working Directory and Project Folders

When Python reads or writes a file (such as saving a graph or loading a lab dataset), it looks in its **Current Working Directory (CWD)**.

In Spyder, look at the top-right toolbar. You will see a folder path with an open folder icon next to it:

```
[ Directory Path: /home/user/College_Python/Lab1 ]  [ Browse]
```

Best Practice for College Labs:

1. Create a dedicated folder on your computer for your course, e.g., `Documents/Python_Labs/`.
2. For every assignment or chapter, create a subfolder: `Chapter00/`, `Chapter01/`, etc.
3. In Spyder, click the **Browse folder icon ( )** and select that folder.
4. Alternatively, execute this in the IPython console:

```
import os
os.chdir("/path/to/your/folder")  # Sets the current directory
print("Current working folder:", os.getcwd())
```

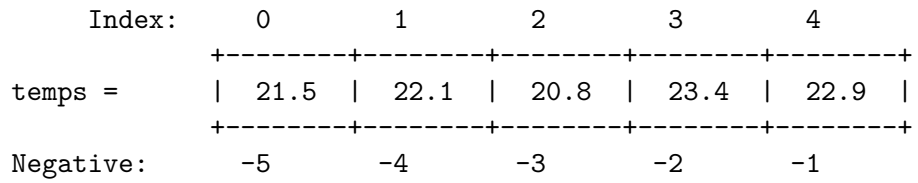
You are now fully set up with Anaconda and Spyder. In the next chapter, we dive into Python's most fundamental data structure: **Lists**.

Core Data Structures and Functions

Lists

Chapter 1: Lists

In scientific computing, we rarely work with isolated numbers. We record sequences of temperatures over time, vectors of spatial coordinates, and tables of experimental readings. In Python, the **list** is the foundational tool for holding ordered collections of data.



1.1 Initializing Lists and Data Types

A list is created by enclosing comma-separated values in square brackets `[...]`. Unlike arrays in languages like C or Java, a single Python list can hold items of different data types simultaneously.

Runnable Example

```
# A list of sensor measurements (all floats)
temperatures = [21.5, 22.1, 20.8, 23.4, 22.9]

# A list of mixed types: [Sample ID, Sensor Name, Active Flag, Reading, Error Count]
sensor_record = [101, "Thermocouple-A", True, 98.6, 0]

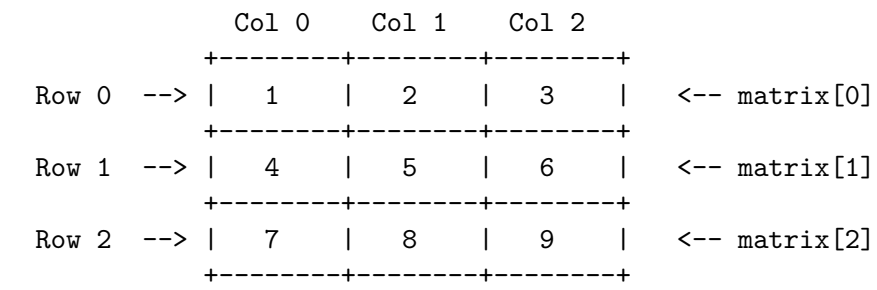
print("Temperatures:", temperatures)
print("Sensor record:", sensor_record)
print("Type of sensor_record:", type(sensor_record))
```

Output:

```
Temperatures: [21.5, 22.1, 20.8, 23.4, 22.9]
Sensor record: [101, 'Thermocouple-A', True, 98.6, 0]
Type of sensor_record: <class 'list'>
```

1.2 1D and 2D Lists (Lists of Lists)

When data represents a grid, table, or matrix, we nest lists inside another list.



Runnable Example

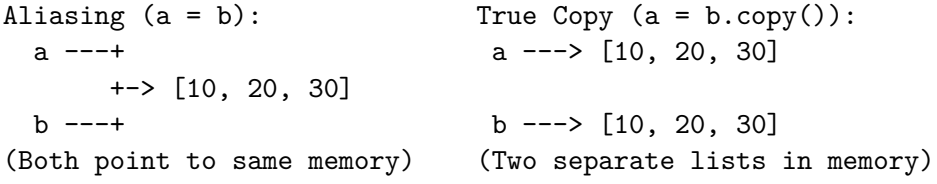
```
# 1D List (a mathematical vector)
vector = [3.0, -1.5, 4.2]

# 2D List (a 3x3 matrix)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print("Row 0:", matrix[0])
print("Element at row 1, col 2:", matrix[1][2])  # 6
```

1.3 Mutability: In-Place Changes, Aliasing vs Copying

Lists are **mutable**, meaning you can modify their contents after creation without creating a new list. However, this introduces the concept of **aliasing**—a frequent source of bugs for beginners!



Runnable Example

```
# 1. In-place modification
numbers = [10, 20, 30]
numbers[1] = 99
print("Modified list:", numbers)  # [10, 99, 30]

# 2. Aliasing trap (Caution!)
original = [1, 2, 3]
alias = original         # alias points to the EXACT SAME object in memory
alias[0] = 500
print("Original after alias change:", original)  # [500, 2, 3]!

# 3. Proper copying
safe_original = [1, 2, 3]
safe_copy = safe_original.copy()   # Or safe_original[:]
safe_copy[0] = 999
print("Safe original stays untouched:", safe_original)  # [1, 2, 3]
print("Copy was modified:", safe_copy)                   # [999, 2, 3]
```

`.copy()` duplicates a flat list. A list of lists still shares its inner rows after `.copy()`, so changing a row in the copy changes the original. Use `copy.deepcopy` for a fully separate nested list.

1.4 `len()` for Lists and Sublists

The `len()` function returns the number of items. For nested lists, `len()` on the outer list gives the number of rows, while `len()` on an individual element gives the number of columns in that row.

Runnable Example

```
grid = [
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
]

num_rows = len(grid)
num_cols = len(grid[0])

print("Number of rows:   ", num_rows)  # 3
print("Number of columns:", num_cols)  # 4
print("Total cells:      ", num_rows * num_cols)
```

1.5 Extracting Elements: Indexing and Slicing

Python uses **0-based indexing**. Slicing syntax is `list[start:stop:step]`, where: - `start` is included. - `stop` is excluded. - `step` is the stride (optional, defaults to 1).

```
List values:  [ 'A',  'B',  'C',  'D',  'E',  'F' ]
Positive idx:    0     1     2     3     4     5
Negative idx:   -6    -5    -4    -3    -2    -1

Slice [1:4]  ------>  [ 'B',  'C',  'D' ]
```

Runnable Example

```
data = [10, 20, 30, 40, 50, 60, 70]

# Single element indexing
print("First element (idx 0): ", data[0])
print("Last element (idx -1): ", data[-1])
print("Second to last:        ", data[-2])

# Slicing
print("Elements 1 to 4:       ", data[1:5])    # [20, 30, 40, 50]
print("First 3 elements:      ", data[:3])     # [10, 20, 30]
print("From index 3 to end:   ", data[3:])     # [40, 50, 60, 70]
print("Every second element:  ", data[::2])    # [10, 30, 50, 70]
print("Reversed list:         ", data[::-1])   # [70, 60, 50, 40, 30, 20, 10]
```

1.6 Iterating with a `for` Loop

You can loop directly over elements or loop using indices with `enumerate()`.

Runnable Example

```
voltages = [1.2, 2.4, 3.6, 4.8]

print("--- Direct iteration ---")
for v in voltages:
    print("Voltage reading:", v, "V")
```

```
print("\n--- Indexed iteration with enumerate ---")
for i, v in enumerate(voltages):
    print(f"Sample {i}: {v} V")
```

1.7 `range()`: Syntax, Usage, and Limitations

The `range(start, stop, step)` function generates a sequence of integers on demand without storing them all in memory.

```
range(2, 10, 2)  --> produces 2, 4, 6, 8 (stops BEFORE 10)
```

> **Important Limitation:** `range()` only accepts **integers**. It cannot generate float steps like `0.1` directly (for floats, we use NumPy or a list comprehension).

Runnable Example

```
# Counting 0 to 4
for i in range(5):
    print(i, end=" ")
print()

# Counting 10 to 30 by 5s
for val in range(10, 31, 5):
    print(val, end=" ")
print()
```

1.8 List Comprehensions for 1D and 2D Lists

List comprehensions offer a compact, readable syntax for creating lists from existing sequences.

Syntax: [expression **for** item **in** iterable]

Runnable Example

```
# 1D List Comprehension: Square of numbers 1 to 6
squares = [x**2 for x in range(1, 7)]
print("Squares:", squares)  # [1, 4, 9, 16, 25, 36]

# With a condition (filtering): only even squares
even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print("Even squares:", even_squares)

# 2D list comprehension: 3x3 matrix of zeros
zeros_2d = [[0 for col in range(3)] for row in range(3)]
print("2D zero grid:", zeros_2d)
```

1.9 Creating a List of Floats Using `range()`

Since `range()` only works with integers, we can divide the generated integers inside a list comprehension to obtain floating-point sequences:

Runnable Example

```
# Create time points from 0.0 to 1.0 with a step of 0.1
# Step = 0.1 = 1 / 10
time_points = [i / 10.0 for i in range(11)]
print("Time points (seconds):")
print(time_points)
```

Output:

```
Time points (seconds):
[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
```

1.10 Converting to a List with `list()`

Any iterable object (tuples, strings, ranges, dictionary keys) can be converted into a list using the `list()` constructor:

```
# Convert range to list
r_list = list(range(5))

# Convert string to list of characters
char_list = list("PYTHON")

print("Range as list: ", r_list)      # [0, 1, 2, 3, 4]
print("String as list:", char_list)  # ['P', 'Y', 'T', 'H', 'O', 'N']
```

Spyder Tip: Variable Explorer

Run the script containing `matrix` in Spyder (`F5`). Open the **Variable Explorer** pane and look for `matrix`. Double-click on it! A dedicated dialog opens, displaying your 2D list in an interactive spreadsheet format with columns and rows labeled clearly.

Exercises and Mini-Lab

Exercise 1: Row and Column Sums

Write a script that takes a 2D matrix of numbers and calculates: 1. The sum of each row. 2. The sum of each column.

```
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]

# Calculate row sums
row_sums = [sum(row) for row in matrix]
print("Row sums:", row_sums)  # [60, 150, 240]

# Calculate column sums
num_cols = len(matrix[0])
col_sums = [
    sum(matrix[r][c] for r in range(len(matrix)))
    for c in range(num_cols)
]
print("Column sums:", col_sums)  # [120, 150, 180]
```

Mini-Lab: Multiplication Table Generator

Generate a 10×10 multiplication table using a nested list comprehension and print it nicely formatted:

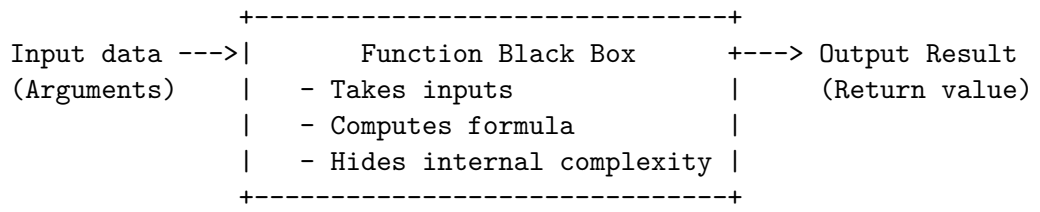
```
# Generate table
table = [[row * col for col in range(1, 11)] for row in range(1, 11)]

# Display formatted table
print("      " + " ".join(f"{c:4d}" for c in range(1, 11)))
print("    +" + "-" * 45)
for r_idx, row in enumerate(table, start=1):
    row_str = " ".join(f"{val:4d}" for val in row)
    print(f"{r_idx:2d}  | {row_str}")
```

Functions

Chapter 2: Functions

As your programs grow, copy-pasting code leads to bugs, repetition, and headaches. **Functions** are self-contained mini-programs that accept inputs, perform specific calculations, and return results. They form the building blocks of clean scientific code.



2.1 Why Functions?

In science and engineering, functions provide three essential advantages: 1. **Repeatability:** Write the code once and use it thousands of times with different inputs. 2. **Decomposition:** Break a massive, confusing problem into small, manageable, testable steps. 3. **Abstraction:** You don't need to know how `math.sqrt()` computes a square root internally to use it effectively.

2.2 Defining a Function and `return`

A function is defined with the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. The indented block is the body. The `return` statement sends a result back to the caller and **immediately terminates** execution of the function.

```
def function_name(param1, param2):
    # Function body (indented)
    result = param1 + param2
    return result   <-- Sends value back and exits
```

Runnable Example

```
def kinetic_energy(mass, velocity):
    """Calculate kinetic energy in Joules: KE = 0.5 * m * v^2"""
    ke = 0.5 * mass * (velocity ** 2)
    return ke

# Calling the function
car_ke = kinetic_energy(1200.0, 25.0)  # 1200 kg car at 25 m/s
print("Kinetic energy:", car_ke, "Joules")
```

If a function does not have a `return` statement (or reaches the end without one), it implicitly returns `None`.

2.3 Formal vs Actual Parameters

- **Formal Parameters:** The variable names listed in the function definition (the placeholders).
- **Actual Parameters (Arguments):** The real values or variables passed into the function when you call it.

```
# formal parameters: voltage, resistance
def ohms_law(voltage, resistance):
    return voltage / resistance

# actual arguments: 12.0 and 4.0
current = ohms_law(12.0, 4.0)
```

2.4 Positional, Keyword Arguments, and Defaults

Python gives you flexibility in how arguments are passed: - **Positional arguments:** Matched by position (first to first, second to second). - **Keyword arguments:** Explicitly matched by name (`param=value`), regardless of order. - **Default arguments:** Provide fallback values if the caller omits them.

Runnable Example

```
def radioactive_decay(initial_amount, time, half_life=5.0):
    """
    Computes remaining substance: N(t) = N0 * (0.5)^(t / half_life).
    half_life defaults to 5.0 time units if omitted.
    """
    remaining = initial_amount * (0.5 ** (time / half_life))
    return remaining

# 1. Positional call using default half_life (5.0)
print(
    "Amount left after 10s (default half-life):",
    radioactive_decay(100.0, 10.0),
)

# 2. Positional call overriding half_life
print(
    "Amount left after 10s (half-life=2.0):",
    radioactive_decay(100.0, 10.0, 2.0),
)

# 3. Keyword call (order does not matter)
print("Keyword call:", radioactive_decay(
    time=15.0, initial_amount=80.0, half_life=3.0))
```

2.5 The * Operator: *args and Unpacking

When you do not know in advance how many arguments will be passed, use `*args`. The `*` packs multiple positional arguments into a single tuple.

```
Call: average(10, 20, 30, 40)
           |   |   |   |
           v   v   v   v
Function receives args = (10, 20, 30, 40)
```

Runnable Example

```
def compute_mean(*numbers):
    """Compute the arithmetic mean of any number of inputs."""
    if len(numbers) == 0:
        return 0.0
```

```
    total = sum(numbers)
    return total / len(numbers)

print("Mean of 3 values:", compute_mean(10, 20, 30))
print("Mean of 5 values:", compute_mean(2.5, 3.5, 4.0, 1.0, 9.0))

# Unpacking a list into arguments using *
sample_data = [12.0, 15.5, 18.2, 14.1]
print("Mean from unpacked list:", compute_mean(*sample_data))
```

2.6 Lambda Functions (Anonymous Functions)

A **lambda** is a small, one-line function defined without a name:

Syntax: **lambda** arguments : expression

Runnable Example

```
# Standard function vs Lambda function
def standard_square(x):
    return x ** 2

lambda_square = lambda x: x ** 2

print("Standard:", standard_square(5))  # 25
print("Lambda:  ", lambda_square(5))    # 25

# Practical use: sorting a list of (x, y) coordinates by y-value
points = [(1, 9), (4, 2), (3, 7), (8, 1)]
sorted_by_y = sorted(points, key=lambda p: p[1])
print("Points sorted by y-coordinate:", sorted_by_y)
```

2.7 Variable Scope and the Stack Frame

Variables created inside a function are **local** to that function. They live inside the function's **stack frame** and are destroyed when the function finishes executing.

```
Global Scope:
+---------------------------------+
| x = 100                         |
|                                 |
|   Stack Frame: calculate(a=5)   |
|   +-------------------------+   |
|   | a = 5                   |   |
|   | temp = a * 2 = 10       |   |
|   +-------------------------+   |
|   (Destroyed after return!)     |
+---------------------------------+
```

Runnable Example

```python
x = 100  # Global variable

def calculate(a):
    temp = a * 2  # Local variable
    print("Inside calculate(): temp =", temp, "and global x =", x)
    return temp

result = calculate(5)
print("Result:", result)

# Trying to access 'temp' here will cause a NameError:
# print(temp)  # Error: name 'temp' is not defined!
```

2.8 Docstrings and Spyder's Help Pane

A **docstring** is a multi-line string placed immediately below the function header. It documents what the function does, its parameters, and what it returns.

```python
def celsius_to_kelvin(temp_c):
    """
    Convert temperature from degrees Celsius to Kelvin.

    Parameters
    ----------
    temp_c : float
        Temperature in Celsius (°C).

    Returns
```

```
    -------
    float
        Temperature in Kelvin (K).
    """
    return temp_c + 273.15
```

Spyder Tip: Interactive Help

In Spyder, type `celsius_to_kelvin(` in the Editor or Console and press `Ctrl + I` (`Cmd + I` on Mac). The **Help Pane** will automatically render your docstring beautifully formatted!

2.9 The `global` Keyword

To modify a global variable from inside a function, Python requires the `global` keyword.

> **Advice for Beginners:** Use `global` sparingly! Functions that modify global variables are harder to test and debug. Prefer returning values instead.

Runnable Example

```
total_samples = 0  # Global counter

def record_measurement(value):
    global total_samples
    total_samples += 1
    print(f"Sample #{total_samples} recorded: {value} V")

record_measurement(3.3)
record_measurement(3.4)
print("Total samples logged:", total_samples)  # 2
```

2.10 Recursion: Base Case and Call Stack

A **recursive function** is a function that calls itself to solve a smaller version of the problem. Every recursive function **must** have: 1. **Base Case:** A stopping condition that returns directly without further recursion. 2. **Recursive Step:** Where the function calls itself with a reduced input.

```
factorial(3)
  +-- 3 * factorial(2)
        +-- 2 * factorial(1)
              +-- 1  (Base case: returns 1)
```

Runnable Example: Factorial

```python
def factorial(n):
    """Compute n! recursively. n is a non-negative integer."""
    # Base case. Both 0! and 1! are 1.
    if n == 0 or n == 1:
        return 1
    # Recursive case
    return n * factorial(n - 1)

print("5! =", factorial(5))  # 5 * 4 * 3 * 2 * 1 = 120
```

A negative `n` never hits that base case. Python then stops the calls with `RecursionError`.

2.11 Functions as First-Class Objects

In Python, functions are objects! You can assign them to variables, store them in lists, and pass them as arguments to other functions (often called *higher-order functions*).

Runnable Example

```python
import math

def evaluate_at_points(func, points):
    """Applies a mathematical function 'func' to a list of x values."""
    return [func(x) for x in points]

x_values = [0.0, math.pi / 4, math.pi / 2, math.pi]

# Pass math.sin as an argument
sin_values = evaluate_at_points(math.sin, x_values)
print("Sin values:", [round(v, 4) for v in sin_values])

# Pass a custom lambda as an argument
squared_values = evaluate_at_points(lambda x: x**2, [1, 2, 3, 4, 5])
print("Squares:   ", squared_values)
```

Exercises and Mini-Lab

Exercise 1: Fibonacci Sequence (Iterative vs Recursive)

```
# 1. Recursive Fibonacci
def fib_recursive(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    return fib_recursive(n - 1) + fib_recursive(n - 2)

# 2. Iterative Fibonacci (much faster for large n!)
def fib_iterative(n):
    if n <= 0:
        return 0
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

print("10th Fibonacci (recursive):", fib_recursive(10))  # 55
print("10th Fibonacci (iterative):", fib_iterative(10))  # 55
```

Mini-Lab: Mathematical Function Table Helper

Write a function `generate_function_table(func, name, x_start, x_end, steps)` that prints a clean text table:

```
def generate_function_table(func, name, x_start, x_end, steps):
    step_size = (x_end - x_start) / (steps - 1)
    print(f"--- Table for {name} ---")
    print(f"{'x':>10} | {'f(x)':>12}")
    print("-" * 25)
    for i in range(steps):
        x = x_start + i * step_size
        y = func(x)
        print(f"{x:10.4f} | {y:12.4f}")

# Test with a cubic polynomial: f(x) = x^3 - 2x + 1
poly = lambda x: x**3 - 2*x + 1
generate_function_table(poly, "x^3 - 2x + 1", -2.0, 2.0, 9)
```

Modules

Chapter 3: Modules

A **module** is simply a Python file (`.py`) containing function definitions, constants, and variables. Modules allow you to organize code logically across multiple files, share tools across different lab assignments, and tap into Python's vast ecosystem of pre-written libraries.

```
+-------------------------------------------+
| main.py                                   |
|   import mytools                          |
|   out = mytools.calculate_stats(data)     |
+-------------------------------------------+
| mytools.py                                |
|   def calculate_stats(data):              |
|       return mean, std_dev                |
+-------------------------------------------+
```

3.1 Importing Modules: The Three Syntax Forms

Python provides three ways to bring code from a module into your script:

```
1. Full import:
   import math
   math.sqrt(16)

2. Alias import (standard in data science):
   import numpy as np
   np.zeros(5)

3. Direct member import:
   from math import sqrt, pi
   sqrt(16)
```

Runnable Example

```
# Style 1: Standard import
import math
print("Sine of pi/2:", math.sin(math.pi / 2))

# Style 2: Import with an alias (shorthand)
import math as m
print("Cosine of 0: ", m.cos(0))

# Style 3: Selective import (import specific functions directly)
from math import sqrt, exp
print("Square root: ", sqrt(25.0))
print("Exponential: ", exp(1.0))
```

> **College Tip:** Avoid `from math import *` (wildcard imports)! It pollutes your namespace with hundreds of names and can overwrite your own variables by accident.

3.2 Importing Part of a Module

When a module is large or hierarchical (e.g., `matplotlib.pyplot` or `scipy.optimize`), you can import just the submodule you need:

```
# Import only the sub-package
import matplotlib.pyplot as plt

# Or specific constants
from math import pi, e
print(f"pi = {pi:.4f}, e = {e:.4f}")
```

3.3 Module-Name Rules and Abstraction

Modules enforce **namespace isolation**. If you define your own variable named `pi = 3.14` and import `math`, you can access both without collision:

```
pi = 3.14           # Your rough local approximation
import math         # Official math module

print("My local pi:    ", pi)
```

```
print("Math module pi: ", math.pi)
```

3.4 The Built-in `math` Module: Common Functions

The standard library includes `math` for scalar calculations. Here are the most frequently used scientific functions:

Function	Description	Example	Result
`math.sqrt(x)`	Square root $\sqrt{x}$	`math.sqrt(9)`	`3.0`
`math.pow(x, y)`	x^y	`math.pow(2, 3)`	`8.0`
`math.sin(x)`, `cos(x)`	Trig functions (radians!)	`math.sin(math.pi/2)`	`1.0`
`math.radians(deg)`	Degrees to radians	`math.radians(180)`	`3.14159...`
`math.degrees(rad)`	Radians to degrees	`math.degrees(math.pi)`	`180.0`
`math.log(x)`	Natural logarithm ($\ln x$)	`math.log(math.e)`	`1.0`
`math.log10(x)`	Base-10 logarithm	`math.log10(100)`	`2.0`
`math.ceil(x)`	Round up to integer	`math.ceil(4.2)`	`5`
`math.floor(x)`	Round down to integer	`math.floor(4.8)`	`4`

3.5 Module Structure: `if __name__ == "__main__"`

When Python runs a file directly, it sets a special built-in variable `__name__` to `"__main__"`. But when that file is **imported** by another script, `__name__` is set to the file's actual module name.

We use this construct to include **testing code** in a module that only runs when the module file is executed on its own, not when it is imported elsewhere!

```
Run the file directly:  __name__ is "__main__"  --> tests run
Import the file:         __name__ is "mytools"   --> tests are skipped
```

3.6 Creating Your Own User-Defined Module

Let us create a reusable module for college physics formulas.

Step 1: Create `physics_tools.py`

In Spyder, create a new file named `physics_tools.py` in your working directory:

```
# physics_tools.py - Reusable physics utility functions
import math

GRAVITY = 9.80665  # m/s^2 on Earth surface

def free_fall_distance(time):
    """Calculate distance fallen under gravity: d = 0.5 * g * t^2"""
    return 0.5 * GRAVITY * (time ** 2)

def pendulum_period(length):
    """Calculate period of simple pendulum: T = 2 * pi * sqrt(L / g)"""
    return 2.0 * math.pi * math.sqrt(length / GRAVITY)

# Self-test block: runs ONLY if you run physics_tools.py directly
if __name__ == "__main__":
    print("Testing physics_tools module:")
    print("Drop distance in 3 seconds: ", free_fall_distance(3.0), "m")
    print("Period of 1m pendulum:      ", pendulum_period(1.0), "s")
```

Step 2: Import It in a Separate Script `lab_experiment.py`

Now create another file in the same folder:

```
# lab_experiment.py
import physics_tools as phys

drop_time = 2.5
dist = phys.free_fall_distance(drop_time)
print(f"Object fell {dist:.2f} meters in {drop_time} seconds.")
```

When you run `lab_experiment.py`, the test printouts from `physics_tools.py` will **not** appear!

3.7 The Module Search Path & Spyder Import Errors

When you type `import mytools`, Python searches for `mytools.py` in this order: 1. **The current working directory** (where your script or Spyder console is pointed). 2. The installed standard library and Conda environment packages. 3. Folders listed in `sys.path`.

```
If you see:  ModuleNotFoundError: No module named 'physics_tools'

Cause:  Spyder's working directory is not set to the folder where
        physics_tools.py is saved!
Fix:    Click the folder icon in Spyder's top-right toolbar and select
        your project folder, or run:
        import os; os.chdir("/path/to/your/folder")
```

Exercises and Mini-Lab

Exercise: Personal Statistics Module (`my_stats.py`)

Create a module named `my_stats.py` with three functions: `mean()`, `variance()`, and `standard_deviation()`.

```python
# my_stats.py
import math

def mean(data):
    """Compute arithmetic average."""
    if not data:
        return 0.0
    return sum(data) / len(data)

def variance(data):
    """Compute sample variance: sum((x - mean)^2) / (N - 1)"""
    if len(data) < 2:
        return 0.0
    m = mean(data)
    sq_diff = sum((x - m) ** 2 for x in data)
    return sq_diff / (len(data) - 1)

def standard_deviation(data):
    """Compute sample standard deviation."""
    return math.sqrt(variance(data))

if __name__ == "__main__":
    test_scores = [85, 92, 78, 90, 88]
    print("Test mean:  ", mean(test_scores))
    print("Test std-dev:", standard_deviation(test_scores))
```

Milestone Project 1: Utility Toolkit

Milestone Project 1: Scientific Utility Toolkit

Congratulations on completing Part 1! In this milestone project, you will build a complete, professional, modular toolkit named **sci_toolkit.py** containing mathematical, statistical, matrix, and recursive helpers. You will then write a test script, **run_toolkit_tests.py**, that imports your module and runs a verification suite.

```
+-------------------------------------------------------------------------+
|                        PROJECT ARCHITECTURE                             |
|                                                                         |
|  [ File: sci_toolkit.py ]              [ File: run_toolkit_tests.py ]   |
|  - Statistics Helpers                  - Imports sci_toolkit            |
|    * mean(), median(), variance()      - Feeds sample lab measurements  |
|  - Matrix Helpers                      - Runs recursive factorials      |
|    * row_totals(), column_totals()     - Displays formatted summary     |
|    * transpose()                                                        |
|  - Recursive Functions                                                  |
|    * recursive_power(base, exp)                                         |
|    * recursive_gcd(a, b)                                                |
+-------------------------------------------------------------------------+
```

---

1. Module Source Code: `sci_toolkit.py`

Save this file in your working folder as `sci_toolkit.py`:

```
"""
sci_toolkit.py
==============
A collection of scientific computing utilities for statistics,
matrix transformations, and recursive algorithms.
"""

import math


# --- Part A: Statistical Helpers ---

def mean(data):
```

```
    """
    Compute the arithmetic mean of a numerical list.

    Parameters
    ----------
    data : list of float or int
        Sequence of numbers.

    Returns
    -------
    float
        The average value.
    """
    if not data:
        return 0.0
    return sum(data) / len(data)


def median(data):
    """
    Compute the median (middle value) of a list of numbers.
    """
    if not data:
        return 0.0
    sorted_data = sorted(data)
    n = len(sorted_data)
    mid = n // 2
    if n % 2 == 1:
        return float(sorted_data[mid])
    else:
        return (sorted_data[mid - 1] + sorted_data[mid]) / 2.0


def variance(data):
    """
    Calculate sample variance: sum((x - mean)^2) / (N - 1).
    """
    if len(data) < 2:
        return 0.0
    m = mean(data)
    return sum((x - m) ** 2 for x in data) / (len(data) - 1)


# --- Part B: Matrix Helpers (Lists of Lists) ---

def matrix_dimensions(matrix):
    """Return (rows, cols) of a 2D matrix."""
```

```
    if not matrix or not matrix[0]:
        return (0, 0)
    return (len(matrix), len(matrix[0]))


def row_totals(matrix):
    """Calculate the sum of each row in a 2D list."""
    return [sum(row) for row in matrix]


def column_totals(matrix):
    """Calculate the sum of each column in a 2D list."""
    if not matrix or not matrix[0]:
        return []
    num_cols = len(matrix[0])
    return [sum(matrix[r][c] for r in range(len(matrix))) for c in range(num_cols)]


def transpose(matrix):
    """
    Transpose a 2D matrix (swap rows and columns).
    Uses a clean nested list comprehension.
    """
    if not matrix or not matrix[0]:
        return []
    num_rows = len(matrix)
    num_cols = len(matrix[0])
    return [[matrix[r][c] for r in range(num_rows)] for c in range(num_cols)]


# --- Part C: Recursive Helpers ---

def recursive_power(base, exponent):
    """Compute base^exponent recursively for non-negative integers."""
    if exponent == 0:
        return 1.0
    elif exponent < 0:
        return 1.0 / recursive_power(base, -exponent)
    return base * recursive_power(base, exponent - 1)


def recursive_gcd(a, b):
    """Euclidean algorithm for greatest common divisor: gcd(a, b)."""
    if b == 0:
        return a
    return recursive_gcd(b, a % b)
```

```
# --- Self-Testing Guard ---
if __name__ == "__main__":
    print("Running internal self-check for sci_toolkit.py...")
    test_data = [10.0, 20.0, 30.0, 40.0, 50.0]
    print("Internal Mean test:", mean(test_data), "(Expected: 30.0)")
```

---

2. Test Driver Script: `run_toolkit_tests.py`

Save this second file in the **same directory** and execute it with `F5` in Spyder:

```
"""
run_toolkit_tests.py
====================
Main program that imports and verifies sci_toolkit.
"""

import sci_toolkit as tk

print("=" * 60)
print("     SCIENTIFIC UTILITY TOOLKIT VERIFICATION SUITE")
print("=" * 60)

# 1. Testing Statistics
sensor_readings = [23.4, 25.1, 22.8, 24.6, 23.9, 24.2, 25.0]
print("\n--- 1. Statistics on Sensor Readings ---")
print("Dataset:      ", sensor_readings)
print(f"Mean:          {tk.mean(sensor_readings):.3f}")
print(f"Median:        {tk.median(sensor_readings):.3f}")
print(f"Variance:      {tk.variance(sensor_readings):.4f}")

# 2. Testing 2D Matrices
sensor_grid = [
    [12, 15, 18],
    [20, 22, 24],
    [30, 31, 35],
    [40, 42, 44]
]
rows, cols = tk.matrix_dimensions(sensor_grid)
print(f"\n--- 2. Matrix Utilities (Grid: {rows}x{cols}) ---")
print("Original Matrix:")
for row in sensor_grid:
    print("  ", row)
```

```
print("Row sums:      ", tk.row_totals(sensor_grid))
print("Column sums:   ", tk.column_totals(sensor_grid))

grid_t = tk.transpose(sensor_grid)
t_rows, t_cols = tk.matrix_dimensions(grid_t)
print(f"Transposed Matrix ({t_rows}x{t_cols}):")
for row in grid_t:
    print("  ", row)

# 3. Testing Recursive Helpers
print("\n--- 3. Recursive Functions ---")
base, exp = 3.0, 4
print(f"{base}^{exp} = {tk.recursive_power(base, exp)} (Expected: 81.0)")

num1, num2 = 48, 18
print(f"GCD({num1}, {num2}) = {tk.recursive_gcd(num1, num2)} (Expected: 6)")

print("\n" + "=" * 60)
print("     ALL TOOLKIT CHECKS COMPLETED SUCCESSFULLY")
print("=" * 60)
```

Expected Console Output:

```
============================================================
     SCIENTIFIC UTILITY TOOLKIT VERIFICATION SUITE
============================================================

--- 1. Statistics on Sensor Readings ---
Dataset:       [23.4, 25.1, 22.8, 24.6, 23.9, 24.2, 25.0]
Mean:          24.143
Median:        24.200
Variance:      0.7129

--- 2. Matrix Utilities (Grid: 4x3) ---
Original Matrix:
   [12, 15, 18]
   [20, 22, 24]
   [30, 31, 35]
   [40, 42, 44]
Row sums:       [45, 66, 96, 126]
Column sums:    [102, 110, 121]
Transposed Matrix (3x4):
   [12, 20, 30, 40]
   [15, 22, 31, 42]
   [18, 24, 35, 44]

--- 3. Recursive Functions ---
```

```
3.0^4 = 81.0 (Expected: 81.0)
GCD(48, 18) = 6 (Expected: 6)

============================================================
     ALL TOOLKIT CHECKS COMPLETED SUCCESSFULLY
============================================================
```

Numerical Computing with NumPy

NumPy Arrays

Chapter 4: NumPy Arrays

While Python's standard lists are flexible, they are not designed for large-scale scientific calculations. If you try to add two lists, Python concatenates them instead of adding their numbers! Furthermore, Python lists store pointers to individual objects scattered across memory, making them slow for millions of calculations.

NumPy (Numerical Python) provides the **`ndarray`** (n-dimensional array): a lightning-fast, memory-efficient data structure built for mathematics.

```
Python List:  [ ptr ] --> [ Float: 1.2 ]
              [ ptr ] --> [ Float: 2.4 ]   (Scattered in RAM, high overhead)
              [ ptr ] --> [ Float: 3.6 ]

NumPy ndarray:  [ 1.2 | 2.4 | 3.6 ]         (Contiguous block of raw C memory)
```

4.1 Importing NumPy

The universal convention across all scientific and engineering communities is to import NumPy under the alias `np`:

```
import numpy as np
```

4.2 The `ndarray` as the Core Structure

An `ndarray` is a grid of values, all of the **exact same type**, indexed by a tuple of non-negative integers.

Runnable Example

```
import numpy as np

# Create an array from a Python list
arr = np.array([10, 20, 30, 40, 50])

print("Array:", arr)
print("Type: ", type(arr))  # <class 'numpy.ndarray'>
```

4.3 Homogeneity and Multidimensionality

Key properties of NumPy arrays: 1. **Homogeneity:** Every element must have identical data types (e.g., all 64-bit floats or all 32-bit integers). If you mix types, NumPy automatically **upcasts** them to the most general type. 2. **Multidimensionality:** Arrays can have 1 dimension (vectors), 2 dimensions (matrices), 3 dimensions (cubes/time-series grids), or more.

```
1D array (vector), shape (4,)

+---+---+---+---+
| 1 | 2 | 3 | 4 |
+---+---+---+---+

2D array (matrix), shape (2, 3)

+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+

3D array, shape (2, 2, 3): two layers of shape (2, 3)

Layer 0
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+
| 4 | 5 | 6 |
+---+---+---+

Layer 1
+---+---+---+
| 7 | 8 | 9 |
```

```
+---+---+---+
|10 |11 |12 |
+---+---+---+
```

Upcasting in Action

```
mixed = np.array([1, 2.5, "hello"])
print("Mixed array elements converted to strings:")
print(mixed)
print("dtype:", mixed.dtype)  # <U32 (Unicode string)
```

4.4 Creating Arrays and Specifying `dtype`

You can explicitly control memory usage by specifying `dtype`:

Data Type (`dtype`)	Description	Memory per Element
`np.float64` (default float)	Double-precision float	8 bytes
`np.float32`	Single-precision float	4 bytes
`np.int64` (default int)	64-bit integer	8 bytes
`np.int32`	32-bit integer	4 bytes
`np.bool_`	Boolean (`True` or `False`)	1 byte

Runnable Example

```
# Force array to be 32-bit floating point numbers
float_arr = np.array([1, 2, 3], dtype=np.float32)
print("Float32 array:", float_arr)
print("Element dtype:", float_arr.dtype)
```

4.5 Inspecting Arrays: `shape`, `ndim`, and `size`

NumPy arrays carry informative attributes:

```
matrix = np.array([
    [1.0, 2.0, 3.0, 4.0],
    [5.0, 6.0, 7.0, 8.0],
```

```
    [9.0, 10.0, 11.0, 12.0]
])

print("Number of dimensions (ndim):", matrix.ndim)   # 2
print("Dimensions shape (rows, cols):", matrix.shape)  # (3, 4)
print("Total number of elements:    ", matrix.size)   # 12
print("Data type of each item:       ", matrix.dtype)  # float64
print("Bytes per item:               ", matrix.itemsize)  # 8
```

4.6 Initializing with `np.zeros()` and `np.ones()`

In scientific algorithms, we often pre-allocate memory before filling it inside an iterative loop:

```
# A 1D array of five zeros
z1 = np.zeros(5)
print("1D Zeros:\n", z1)

# A 2D (3 rows, 4 columns) array of ones
o2 = np.ones((3, 4), dtype=np.float64)
print("2D Ones (3x4):\n", o2)

# Constant value array using np.full()
f = np.full((2, 3), 9.81)
print("Constant array (gravity):\n", f)
```

4.7 `np.arange()` vs `np.linspace()` (When to Use Which)

Both functions generate sequences of numbers, but they ask different questions:

```
  np.arange(start, stop, step)  --> "I know the STEP size I want."
  np.linspace(start, stop, num) --> "I know HOW MANY points I want in total."

arange(0, 10, 2):
0       2       4       6       8       (Stops BEFORE 10!)
+-------+-------+-------+-------+

linspace(0, 10, 5):
0     2.5       5      7.5      10      (INCLUDES 10! Exactly 5 points)
+-------+-------+-------+-------+
```

Runnable Example

```
# 1. arange: From 0 up to (excluding) 1.0 with step 0.2
t_step = np.arange(0.0, 1.0, 0.2)
print("arange (step=0.2):", t_step)

# 2. linspace: Exactly 5 evenly spaced points between 0.0 and 1.0 (inclusive!)
t_points = np.linspace(0.0, 1.0, 5)
print("linspace (5 pts): ", t_points)
```

> **Rule of Thumb for Plotting:** Always prefer `np.linspace()` when plotting curves (e.g. `np.linspace(0, 2*np.pi, 200)`), because it guarantees clean start and end points without rounding errors.

4.8 Reshaping Arrays with `np.reshape()`

You can change an array's dimensions as long as the total number of elements remains the same ($rows \times cols = size$).

```
1D Vector (12 elements)
[ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 ]
                  |
                  v reshape(3, 4)
[[ 0,  1,  2,  3],
 [ 4,  5,  6,  7],
 [ 8,  9, 10, 11]]
```

Runnable Example

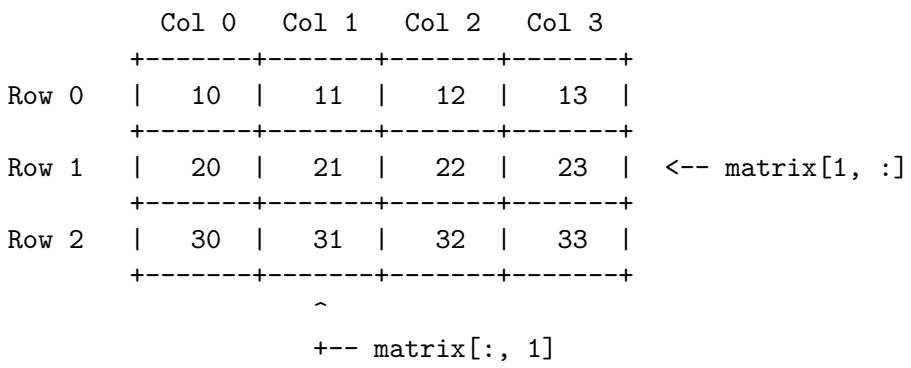
```
linear = np.arange(12)  # [0, 1, ..., 11]

# Reshape into 3 rows, 4 columns
grid_3x4 = linear.reshape(3, 4)
print("Reshaped to 3x4:\n", grid_3x4)

# You can use -1 to let NumPy infer one of the dimensions automatically!
auto_cols = linear.reshape(2, -1)
print("Reshaped to (2, auto) -> (2, 6):\n", auto_cols)
```

4.9 Indexing and Slicing in 1D and 2D

Unlike Python lists which require `matrix[r][c]`, NumPy allows intuitive multi-index comma syntax: `matrix[row_slice, col_slice]`.



Runnable Example

```
m = np.array([
    [10, 11, 12, 13],
    [20, 21, 22, 23],
    [30, 31, 32, 33]
])

# Single element at row 1, col 2
print("Element (1, 2):     ", m[1, 2])     # 22

# Extract entire row 1
print("Entire Row 1:       ", m[1, :])     # [20, 21, 22, 23]

# Extract entire column 2
print("Entire Col 2:       ", m[:, 2])     # [12, 22, 32]

# Sub-matrix: rows 0 to 1, columns 1 to 2
print("Sub-matrix (2x2):\n", m[0:2, 1:3])
```

4.10 Viewing Arrays in Spyder's Variable Explorer

In Spyder: 1. Run any script creating a NumPy array. 2. In the **Variable Explorer** pane, find your array. 3. Notice that Spyder shows its name, type (`ndarray`), shape `(3, 4)`, and min/max values. 4. **Double click on the array:** Spyder opens a visual viewer that color-codes numbers by magnitude (heatmap style), making it easy to identify data patterns, outliers, or zeros at a glance!

Array Methods, Operations and Sorting

Chapter 5: Array Methods, Operations, and Sorting

NumPy's true power lies in **vectorization**: performing complex mathematical operations across thousands or millions of numbers in a single, elegant line of code—without writing a single `for` loop.

```
Loop Approach (Slow):
for i in range(len(a)):
    c[i] = a[i] + b[i]

Vectorized Approach (Fast):
c = a + b   <-- C-level SIMD acceleration under the hood!
```

5.1 Basic Methods: `sum()`, `max()`, `min()`, and `T`

You can call reduction methods either directly on the array object or via the `np` module:

Runnable Example

```python
import numpy as np

arr = np.array([14.2, 8.5, 23.1, 19.4, 5.0])

print("Total sum:        ", arr.sum())
print("Maximum value:    ", arr.max())
print("Minimum value:    ", arr.min())
print("Mean value:       ", arr.mean())
print("Standard deviation:", arr.std())
print("Index of max value:", arr.argmax())
```

`arr.std()` divides by N, which is the population standard deviation. Lab reports usually want the sample standard deviation, `arr.std(ddof=1)`, the same $N - 1$ divisor used for sample variance in Chapter 3.

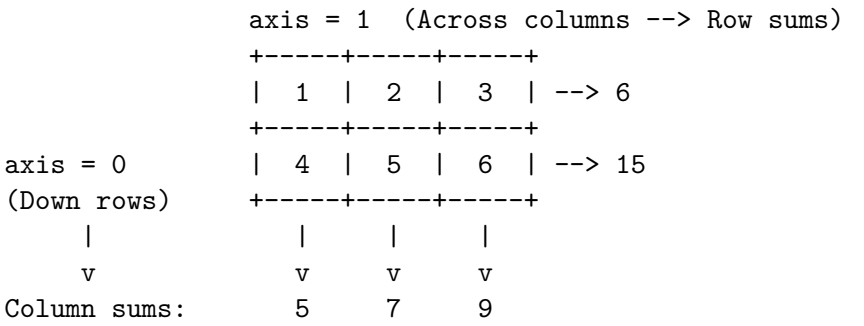
Transposing a Matrix

To swap rows and columns, use the `.T` attribute or `np.transpose()`:

```
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print("Original matrix (2x3):\n", A)
print("Transposed matrix (3x2):\n", A.T)
```

5.2 Operating Over Axes: Rows vs Columns

In 2D arrays, reductions can be calculated across the entire matrix or along a specific **axis**:



> **Memory Rule for 2D Arrays:** - `axis = 0`: Collapses rows (operates down each column). - `axis = 1`: Collapses columns (operates across each row).

Runnable Example

```
grades = np.array([
    [85, 90, 78],  # Student 0 [Exam 1, Exam 2, Exam 3]
    [72, 88, 91],  # Student 1
    [95, 92, 89],  # Student 2
    [60, 75, 70]   # Student 3
])

# Average grade for each student (across exams -> axis=1)
student_averages = grades.mean(axis=1)
print("Student averages (axis=1):", student_averages)

# Average grade for each exam (down students -> axis=0)
```

```
exam_averages = grades.mean(axis=0)
print("Exam averages (axis=0):   ", exam_averages)
```

5.3 Sorting Arrays

NumPy provides in-place sorting via `.sort()` and non-destructive sorting via `np.sort()`:

Runnable Example

```
data = np.array([45, 12, 89, 33, 27])

# Non-destructive sort (returns a new array)
sorted_data = np.sort(data)
print("Sorted array:  ", sorted_data)
print("Original data: ", data)

# 2D sorting along axes
grid = np.array([[3, 1, 4], [1, 5, 9]])
print("Sorted along rows (axis=1):\n", np.sort(grid, axis=1))

# argsort: returns the indices that would sort the array
idx = np.argsort(data)
print("Indices of sorted items:", idx)
```

5.4 Elementwise Mathematical Functions

Standard mathematical functions in NumPy operate on every item in the array simultaneously:

$$\sin(\vec{x}), \quad e^{\vec{x}}, \quad \ln(\vec{x}), \quad \vec{x}^{p}$$

Runnable Example

```
angles_deg = np.array([0.0, 30.0, 45.0, 60.0, 90.0])

# Convert degrees to radians
angles_rad = np.radians(angles_deg)
```

```
# Elementwise trigonometric and exponential functions
sines = np.sin(angles_rad)
exponentials = np.exp(np.array([1, 2, 3]))
logs = np.log(exponentials)  # Natural logarithm

print("Angles (rad):", np.round(angles_rad, 3))
print("sin(theta):  ", np.round(sines, 3))
print("log(exp(x)): ", logs)  # [1. 2. 3.]
```

5.5 Vectorization vs Loops: Speed Benchmark

Why do scientists insist on vectorization? Because vectorized NumPy is written in optimized C/Fortran and runs **10x to 100x faster** than pure Python loops!

```
import time

# Create 1 million numbers
N = 1_000_000
py_list = list(range(N))
np_arr = np.arange(N)

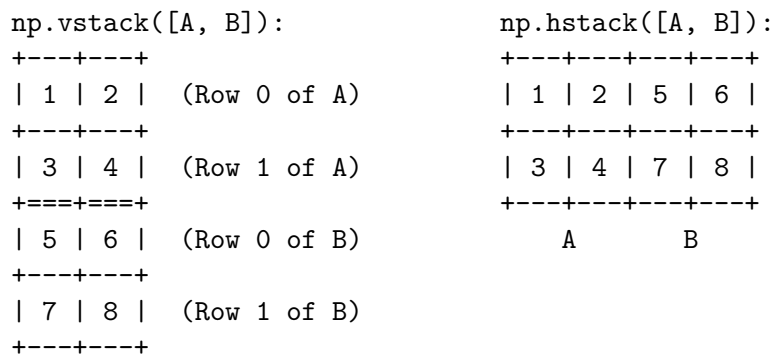
# 1. Pure Python loop
t0 = time.time()
py_result = [x ** 2 for x in py_list]
t_loop = time.time() - t0

# 2. Vectorized NumPy
t0 = time.time()
np_result = np_arr ** 2
t_vector = time.time() - t0

print(f"Python list loop:  {t_loop:.5f} seconds")
print(f"NumPy vectorized:  {t_vector:.5f} seconds")
print(f"NumPy is {t_loop / t_vector:.1f}x faster!")
```

5.6 Sequencing Arrays: `np.vstack()` and `np.hstack()`

You can combine existing arrays vertically (stack rows) or horizontally (stack columns):



Runnable Example

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

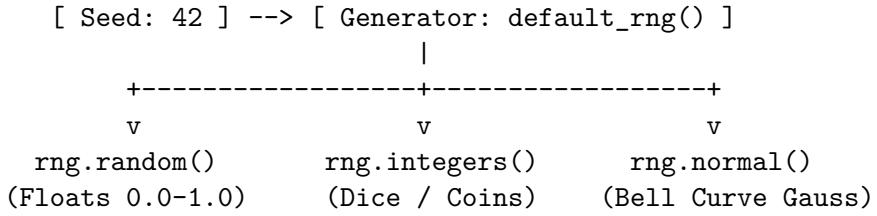
# Vertical stacking: puts b underneath a
v_stacked = np.vstack([a, b])
print("Vertically stacked (4x2):\n", v_stacked)

# Horizontal stacking: puts b to the right of a
h_stacked = np.hstack([a, b])
print("Horizontally stacked (2x4):\n", h_stacked)
```

Random Numbers in NumPy

Chapter 6: Random Numbers in NumPy

From simulating thermal motion of molecules to modeling financial market fluctuations, randomness plays a vital role in scientific computing. NumPy features a state-of-the-art pseudo-random number generator that is both fast and statistically robust.



6.1 Creating a Modern Generator: `default_rng()`

Modern NumPy uses the **`default_rng()`** interface (based on the PCG64 algorithm), replacing the older legacy `np.random.rand()` API.

```
import numpy as np

# Instantiate a fresh random number generator
rng = np.random.default_rng()
```

6.2 Generating Random Floats and Integers

You can generate uniform floats, integers, or samples from scientific probability distributions:

Runnable Example

```
import numpy as np

rng = np.random.default_rng()
```

```
# 1. Random floats in range [0.0, 1.0)
uniform_floats = rng.random(size=5)
print("5 Uniform floats:\n", np.round(uniform_floats, 4))

# 2. Random integers in range [low, high)
# Simulating a standard 6-sided die (from 1 up to 7, exclusive)
dice_rolls = rng.integers(low=1, high=7, size=10)
print("10 Dice rolls:   ", dice_rolls)

# 3. Normally distributed (Gaussian) numbers: mean=0, std=1
bell_curve_samples = rng.normal(loc=0.0, scale=1.0, size=5)
print("5 Normal samples:", np.round(bell_curve_samples, 4))
```

6.3 Seeding for Reproducibility

Scientific experiments must be **repeatable**. If you publish a paper with a simulation, other scientists must be able to verify your results exactly.

Passing a fixed **seed integer** into `default_rng(seed)` guarantees that the exact same sequence of pseudo-random numbers is produced every single run.

```
# Run with seed=42
rng_a = np.random.default_rng(seed=42)
sample_a = rng_a.integers(1, 100, size=5)

# Run again with same seed=42
rng_b = np.random.default_rng(seed=42)
sample_b = rng_b.integers(1, 100, size=5)

print("Sample A:", sample_a)
print("Sample B:", sample_b)
print("Are they identical?", np.array_equal(sample_a, sample_b))  # True!
```

6.4 Scientific Simulations

1. Coin Toss Experiment (Law of Large Numbers)

Simulate flipping a fair coin 100,000 times and calculate the proportion of heads (1 = Head, 0 = Tail):

```python
rng = np.random.default_rng(seed=101)
N = 100_000

# 0 or 1 with equal 50% probability
flips = rng.integers(0, 2, size=N)
heads_count = np.sum(flips == 1)
percentage = (heads_count / N) * 100.0

print(f"Total flips: {N:,}")
print(f"Heads:       {heads_count:,} ({percentage:.2f}%)")
```

2. The 1D Random Walk (Brownian Motion Model)

A particle starts at position $x = 0$. At each time step, it steps either $+1$ or -1 with equal probability. We can model the entire walk instantaneously using `cumsum()` (cumulative sum):

```
Step:         0     1     2     3     4     5
Random delta: -    +1    -1    +1    +1    -1
Position x:   0    +1     0    +1    +2    +1
```

```python
rng = np.random.default_rng(seed=2026)
steps = 10

# Generate +1 or -1 steps
# (rng.choice selects randomly from [-1, 1])
step_deltas = rng.choice([-1, 1], size=steps)

# Cumulative sum computes the position at every instant
positions = np.cumsum(step_deltas)

print("Step choices: ", step_deltas)
print("Trajectory:   ", positions)
print("Final displacement from origin:", positions[-1])
```

Linear Algebra with numpy.linalg

Chapter 7: Linear Algebra with `numpy.linalg`

Linear algebra is the backbone of engineering analysis, quantum mechanics, computer vision, and machine learning. NumPy's `numpy.linalg` submodule bundles high-performance linear algebra routines wrapping industry-standard LAPACK and BLAS libraries.

```
Matrix Equation:   A · x = b

+---------+   +---+   +----+
| 3    1  |   | x |   |  9 |
| 1    2  | · | y | = |  8 |
+---------+   +---+   +----+
     A          x       b
```

7.1 Determinant and Matrix Inverse

For a square matrix A: - **Determinant (`det`):** A scalar reflecting the scaling factor of the linear transformation. If $\det(A) = 0$, the matrix is singular and cannot be inverted. - **Inverse (`inv`):** A^{-1} such that $A \cdot A^{-1} = I$ (Identity matrix).

Runnable Example

```
import numpy as np

A = np.array([
    [4.0, 7.0],
    [2.0, 6.0]
])

# 1. Determinant
det_A = np.linalg.det(A)
print(f"Determinant of A: {det_A:.2f}")  # 4*6 - 7*2 = 10.0

# 2. Matrix Inverse
A_inv = np.linalg.inv(A)
print("Inverse of A:\n", A_inv)
```

```
# A @ A_inv should be the identity, within floating-point roundoff
identity_check = A @ A_inv
print("A @ A_inv:\n", np.round(identity_check, 4))
```

7.2 Eigenvalues and Eigenvectors

Eigenvalues (λ) and eigenvectors ($\vec{v}$) satisfy:

$$A\vec{v} = \lambda\vec{v}$$

They represent directions along which a linear transformation acts merely by stretching or shrinking.

Runnable Example

```
M = np.array([
    [2.0, 1.0],
    [1.0, 2.0]
])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(M)

print("Eigenvalues:       ", eigenvalues)
print("Eigenvectors (columns):\n", eigenvectors)

# Verify for the first eigenvalue: M @ v0 == lambda0 * v0
v0 = eigenvectors[:, 0]
lambda0 = eigenvalues[0]
print("\nM @ v0:      ", M @ v0)
print("lambda0 * v0:", lambda0 * v0)
```

7.3 Solving Systems of Linear Equations

Given a system of linear equations $A\vec{x} = \vec{b}$, find $\vec{x}$.

Never compute $\vec{x} = A^{-1}\vec{b}$ manually in scientific code! Inverting a matrix is slow and numerically unstable. Instead, always use **`np.linalg.solve(A, b)`**, which uses LU decomposition:

$$\begin{cases} 2x + y - z = 8 \\ -3x - y + 2z = -11 \\ -2x + y + 2z = -3 \end{cases}$$

Runnable Example

```
# Coefficient matrix A
A = np.array([
    [ 2.0,  1.0, -1.0],
    [-3.0, -1.0,  2.0],
    [-2.0,  1.0,  2.0]
])

# Right-hand side vector b
b = np.array([8.0, -11.0, -3.0])

# Solve for x
solution = np.linalg.solve(A, b)
print("Solution [x, y, z]:", solution)  # [2., 3., -1.]

# Verify solution by computing A @ x
print("Verification (A @ x):", A @ solution)  # Should match b: [8., -11., -3.]
```

7.4 Types of Products: Dot, Matmul, and `@`

Python provides several ways to multiply arrays:

Operator / Function	Purpose	Example
`*`	**Elementwise** multiplication	`A * B`
`np.dot(u, v)`	Vector dot product / Matrix product	`np.dot(u, v)`
`np.matmul(A, B)` or `@`	**Matrix multiplication**	`A @ B`
`np.inner(u, v)`	Inner product of vectors	`np.inner(u, v)`

Runnable Example

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

# Dot product of two 1D vectors: 1*4 + 2*5 + 3*6 = 32
dot_prod = np.dot(u, v)
print("Vector dot product:", dot_prod)

# Elementwise multiplication vs Matrix multiplication
X = np.array([[1, 2], [3, 4]])
Y = np.array([[2, 0], [1, 2]])

print("Elementwise (X * Y):\n", X * Y)
print("Matrix Product (X @ Y):\n", X @ Y)
```

Exercises and Mini-Lab: Electrical Circuit Analysis

Problem: Kirchhoff's Mesh Current Method

Consider a two-loop DC circuit: - Loop 1 equation: $12I_1 - 4I_2 = 18$ - Loop 2 equation: $-4I_1 + 10I_2 = 0$

Find the loop currents I_1 and I_2 in Amperes.

```
# Resistance matrix (Ohms)
R = np.array([
    [12.0, -4.0],
    [-4.0, 10.0]
])

# Voltage vector (Volts)
V = np.array([18.0, 0.0])

# Solve for currents [I1, I2]
I = np.linalg.solve(R, V)

print(f"Current I1: {I[0]:.3f} A")
print(f"Current I2: {I[1]:.3f} A")

# Check power dissipation: P = I^T * R * I
power = I @ R @ I
print(f"Total Circuit Power: {power:.3f} Watts")
```

Milestone Project 2: Numerical Experiments

Milestone Project 2: Numerical Experiments

In this project, you will build an integrated simulation and analysis script that applies NumPy's computational engine to two classical scientific problems: 1. **2D Particle Random Walk (Brownian Motion Simulation)** — Simulating diffusion over time, computing statistical metrics (mean-squared displacement), and identifying maximum excursions. 2. **Linear Electrical Grid Solver** — Setting up node voltage equations for an electrical grid, solving them with `np.linalg.solve()`, and checking that the solution satisfies the node equations.

```
+-------------------------------------------------------------------------+
|                  MILESTONE 2: COMPUTATIONAL EXPERIMENTS                 |
|                                                                         |
|  [ Experiment 1: 2D Random Walk ]      [ Experiment 2: Linear System ]  |
|  - 10,000 diffusion steps              - 4-node resistor network        |
|  - Tracks X(t) and Y(t)                - Solves KCL Node Voltages       |
|  - Computes radial distance R(t)       - Checks G @ V against I         |
|  - Reports mean squared distance       - Tied to ground, so V is unique |
+-------------------------------------------------------------------------+
```

Complete Project Code: `numerical_experiments.py`

Create a new file in Spyder named `numerical_experiments.py` and run it:

```
"""
numerical_experiments.py
========================
Milestone Project 2: Combining NumPy array operations,
random walks, and linear algebra.
"""

import numpy as np


def run_2d_random_walk(num_steps=10_000, seed=42):
    """
    Simulates a 2D random walk where a particle moves dx, dy ~ N(0, 1)
    at each discrete time interval.
    """
```

```
    print("\n" + "=" * 60)
    print("      EXPERIMENT 1: 2D BROWNIAN MOTION SIMULATION")
    print("=" * 60)

    rng = np.random.default_rng(seed=seed)

    # Generate Gaussian step displacements in x and y
    step_dx = rng.normal(loc=0.0, scale=1.0, size=num_steps)
    step_dy = rng.normal(loc=0.0, scale=1.0, size=num_steps)

    # Prepend origin (0, 0)
    x = np.concatenate(([0.0], np.cumsum(step_dx)))
    y = np.concatenate(([0.0], np.cumsum(step_dy)))

    # Calculate Euclidean distance from origin at each step: R = sqrt(x^2 + y^2)
    radial_distance = np.sqrt(x**2 + y**2)

    final_x = x[-1]
    final_y = y[-1]
    final_r = radial_distance[-1]
    max_r = radial_distance.max()
    max_r_step = radial_distance.argmax()
    mean_sq_disp = np.mean(radial_distance**2)

    print(f"Simulation steps:               {num_steps:,}")
    print(f"Final coordinates (x, y):       ({final_x:.2f}, {final_y:.2f})")
    print(f"Final distance from origin:     {final_r:.2f} units")
    print(
        "Max distance reached:           "
        f"{max_r:.2f} units (at step #{max_r_step:,})"
    )
    print(f"Mean Squared Displacement (MSD): {mean_sq_disp:.2f}")

    return x, y, radial_distance


def solve_electrical_network():
    """
    Solves node voltages in a 4-node DC circuit using Kirchhoff's
    Current Law (KCL): G @ V = I
    where G is the conductance matrix, V is node voltage, and I is injected current.
    """
    print("\n" + "=" * 60)
    print("      EXPERIMENT 2: 4-NODE RESISTOR NETWORK SOLVER")
    print("=" * 60)
```

```
    # Conductance matrix G (siemens). Each diagonal entry includes
    # a conductance to ground, so the voltages have one solution.
    G = np.array([
        [ 0.90, -0.20, -0.50,  0.00],
        [-0.20,  0.95, -0.15, -0.50],
        [-0.50, -0.15,  1.05, -0.30],
        [ 0.00, -0.50, -0.30,  0.90]
    ])

    # Injected currents (Amperes) at each node
    I = np.array([5.0, 0.0, 0.0, -5.0])

    # 1. Check determinant to ensure unique solvability
    det_G = np.linalg.det(G)
    print(f"Conductance Matrix Determinant: {det_G:.4f} (Non-zero -> Solvable)")

    # 2. Solve for Node Voltages V
    V = np.linalg.solve(G, I)

    print("\nCalculated Node Voltages:")
    for node_idx, v_val in enumerate(V, start=1):
        print(f"  Node {node_idx}: {v_val:8.4f} Volts")

    # 3. Verification of Solution: Residual check ||G @ V - I||
    residual = np.linalg.norm(G @ V - I)
    print(f"\nVerification Residual ||G*V - I||: {residual:.2e}")
    if residual < 1e-12:
        print(">> VERIFICATION PASSED: Node equations satisfied to machine precision!")

    return V


if __name__ == "__main__":
    x, y, r = run_2d_random_walk(num_steps=10_000, seed=2026)
    voltages = solve_electrical_network()
```

Expected Console Output:

```
============================================================
      EXPERIMENT 1: 2D BROWNIAN MOTION SIMULATION
============================================================
Simulation steps:               10,000
Final coordinates (x, y):       (-20.73, 30.25)
Final distance from origin:     36.67 units
Max distance reached:           111.74 units (at step #2,880)
Mean Squared Displacement (MSD): 3494.03
```

```
============================================================
      EXPERIMENT 2: 4-NODE RESISTOR NETWORK SOLVER
============================================================
Conductance Matrix Determinant: 0.1936 (Non-zero -> Solvable)

Calculated Node Voltages:
  Node 1:   4.9199 Volts
  Node 2:  -2.6989 Volts
  Node 3:  -0.0646 Volts
  Node 4:  -7.0764 Volts

Verification Residual ||G*V - I||: 0.00e+00
>> VERIFICATION PASSED: Node equations satisfied to machine precision!
```

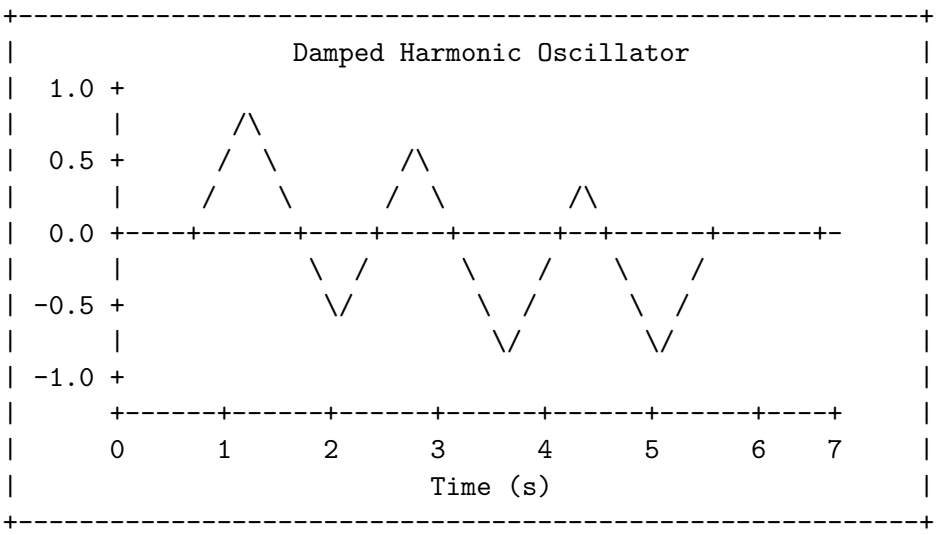
Visualization with Matplotlib

Basic Plotting

Chapter 8: Basic Plotting

A single graph is often worth a thousand tables of raw numbers. In scientific work, visualizing data helps you spot trends, identify outliers, verify theoretical models against experimental measurements, and communicate your findings.

Matplotlib is Python's premier 2D plotting library.



8.1 Importing Matplotlib

The standard convention is to import the `pyplot` submodule as `plt`:

```
import matplotlib.pyplot as plt
import numpy as np
```

8.2 Line Plots, Scatter Plots, and Filled Curves

Matplotlib provides three foundational plotting functions:

1. **`plt.plot(x, y)`:** Connects data points with lines (ideal for continuous mathematical functions and time series).
2. **`plt.scatter(x, y)`:** Displays distinct markers without connecting lines (ideal for discrete experimental lab measurements).
3. **`plt.fill_between(x, y1, y2)`:** Fills the region between two curves with a translucent color (ideal for error bounds or confidence intervals).

Runnable Example

```
import matplotlib.pyplot as plt
import numpy as np

# 1. Theoretical curve
x_model = np.linspace(0, 10, 100)
y_model = 2.5 * x_model + 1.0

# 2. Simulated experimental noisy data
rng = np.random.default_rng(seed=12)
x_data = np.linspace(0.5, 9.5, 12)
noise = rng.normal(0, 1.8, size=len(x_data))
y_data = 2.5 * x_data + 1.0 + noise

# 3. Create visual plot
plt.figure(figsize=(8, 4.5))
plt.plot(x_model, y_model, label="Theoretical Model", color="blue", linewidth=2)
plt.scatter(x_data, y_data, label="Lab Measurements", color="red", marker="o", s=50)

# Fill uncertainty zone (+/- 2 units around model)
plt.fill_between(
    x_model, y_model - 2, y_model + 2,
    color="blue", alpha=0.15, label="Uncertainty Bound",
)

plt.xlabel("Input Current (mA)")
plt.ylabel("Output Voltage (V)")
plt.title("Sensor Calibration Curve")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```

8.3 Styling: Lines, Markers, Colors, and Sizes

You have complete control over styling:

```
plt.plot(x, y, color='green', linestyle='--', linewidth=2.5,
               marker='s', markersize=8, markerfacecolor='yellow')
```

Quick Style Cheat Sheet:

- **Colors:** `'blue'` (`'b'`), `'red'` (`'r'`), `'green'` (`'g'`), `'black'` (`'k'`), `'orange'`, or hex codes (`'#FF5733'`).
- **Line styles:** `'-'` (solid), `'--'` (dashed), `':'` (dotted), `'-.'` (dash-dot).
- **Markers:** `'o'` (circle), `'s'` (square), `'^'` (triangle), `'*'` (star), `'x'` (cross).

8.4 Titles, Labels, and Font Sizes

Always label every axis with its quantity and physical units:

```
plt.title("Pendulum Angle vs Time", fontsize=14, fontweight="bold")
plt.xlabel("Time (seconds)", fontsize=12)
plt.ylabel("Displacement Angle (degrees)", fontsize=12)
```

8.5 Axis Limits and Ticks

Adjust visible axes to focus on key features or add zero margins:

```
x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

plt.plot(x, y)
plt.xlim(0, 2 * np.pi)       # Lock horizontal window from 0 to 2*pi
plt.ylim(-1.2, 1.2)         # Add headroom above/below peak values

# Custom tick marks
plt.xticks([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi],
           ["0", r"$\pi/2$", r"$\pi$", r"$3\pi/2$", r"$2\pi$"])
plt.grid(True)
plt.show()
```

8.6 Legends

When multiple datasets share a plot, pass `label="..."` to each plot command, then call `plt.legend()`:

```
x = np.linspace(0, 4, 100)
plt.plot(x, x, label="Linear: x")
plt.plot(x, x**2, label="Quadratic: x^2")
plt.plot(x, x**3, label="Cubic: x^3")

# location options: 'best', 'upper right', 'lower left', etc.
plt.legend(loc="upper left", frameon=True, shadow=True)
plt.show()
```

8.7 Spyder's Plots Pane

When working in Spyder: - Every plot generated by `plt.show()` appears automatically inside the **Plots Pane** in the upper right. - You can cycle through previously generated figures using the arrow buttons (← / →). - If you prefer floating windows that can be zoomed and panned interactively, type in the IPython console: `python %matplotlib qt # Interactive pop-out window # or to switch back to inline: %matplotlib inline`

8.8 Saving Publication-Ready Figures

Save your figures directly to disk for lab reports, Word documents, or LaTeX papers:

```
# Save as a crisp 300 DPI PNG image
plt.savefig("calibration_curve.png", dpi=300, bbox_inches="tight")

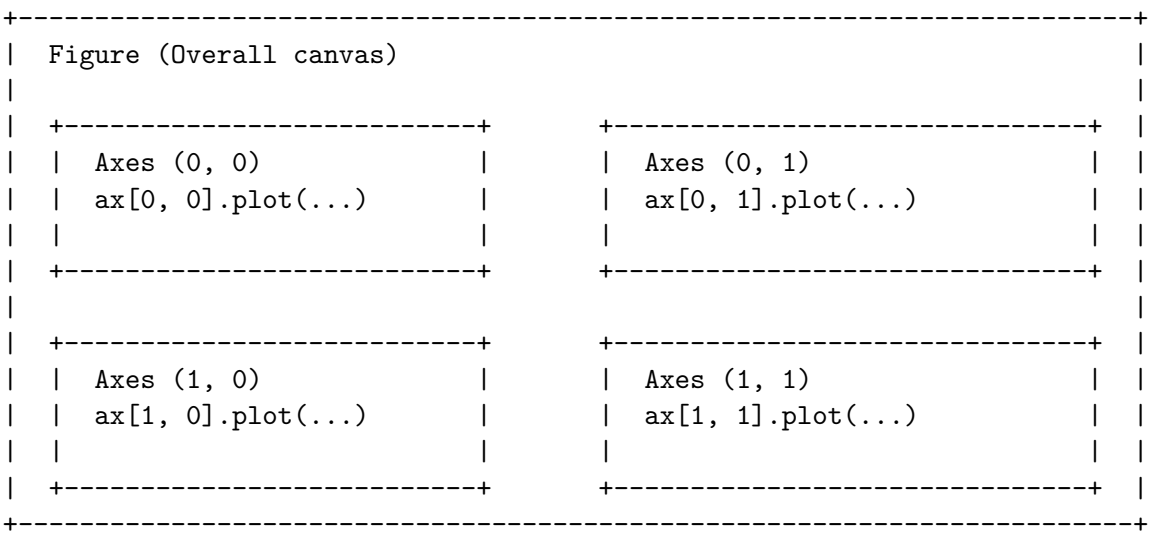
# Or save as vector graphics (infinite resolution PDF/SVG)
plt.savefig("calibration_curve.pdf", bbox_inches="tight")
```

Figures, Axes and Subplots

Chapter 9: Figures, Axes, and Subplots

While quick calls like `plt.plot()` work well for a single graph, professional scientific papers and laboratory reports require multi-panel figures (e.g. comparing raw signal, filtered frequency, phase response, and error distribution in one consolidated image).

To build multi-panel layouts, we transition to Matplotlib's **Object-Oriented (OO) API**.



9.1 The Object-Oriented Approach: `fig, ax`

Think of the **Figure** as the blank poster board and each **Axes** as an individual plot pinned onto that board:

```
fig, ax = plt.subplots(figsize=(7, 4))
```

Instead of calling `plt.title()` or `plt.xlabel()`, in the OO style you call methods on the `ax` object:

State-Machine (`plt`)	Object-Oriented (`ax`)
`plt.title("Title")`	`ax.set_title("Title")`
`plt.xlabel("X")`	`ax.set_xlabel("X")`
`plt.ylabel("Y")`	`ax.set_ylabel("Y")`

State-Machine (`plt`)	Object-Oriented (`ax`)
`plt.xlim(0, 10)`	`ax.set_xlim(0, 10)`
`plt.grid(True)`	`ax.grid(True)`

Runnable Example

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(0, 5, 200)
voltage = 5.0 * np.exp(-t) * np.cos(2 * np.pi * t)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(t, voltage, color="darkblue", linewidth=1.8, label="Damped Response")

# Set properties cleanly in one call using ax.set()
ax.set(title="RLC Circuit Transient Voltage",
       xlabel="Time (seconds)",
       ylabel="Voltage (V)",
       xlim=(0, 5),
       ylim=(-4, 6))

ax.axhline(0, color="gray", linestyle=":", linewidth=1)
ax.grid(True, alpha=0.5)
ax.legend()
plt.show()
```

9.2 Creating Subplot Grids: `plt.subplots(rows, cols)`

Calling `plt.subplots(rows, cols)` returns a 2D NumPy array of Axes objects.

```
# A 2x2 grid of 4 plots
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 8))

# Access specific panels using 2D array indexing:
# axes[0, 0] = Top-Left
# axes[0, 1] = Top-Right
# axes[1, 0] = Bottom-Left
# axes[1, 1] = Bottom-Right
```

9.3 Plotting on Specific Axes

Runnable Example: A 2×2 Scientific Dashboard

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-3, 3, 300)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Panel (0, 0): Sine & Cosine
axes[0, 0].plot(x, np.sin(x), 'b-', label="sin(x)")
axes[0, 0].plot(x, np.cos(x), 'r--', label="cos(x)")
axes[0, 0].set_title("Trigonometric Functions")
axes[0, 0].legend()
axes[0, 0].grid(True)

# Panel (0, 1): Polynomials
axes[0, 1].plot(x, x**2, 'g-', label="x^2")
axes[0, 1].plot(x, x**3, 'm-.', label="x^3")
axes[0, 1].set_title("Power Functions")
axes[0, 1].legend()
axes[0, 1].grid(True)

# Panel (1, 0): Exponential Decay
axes[1, 0].plot(x, np.exp(-x**2), 'darkorange', linewidth=2)
axes[1, 0].set_title("Gaussian Bell Curve: exp(-x^2)")
axes[1, 0].grid(True)

# Panel (1, 1): Histogram of Random Gaussian Noise
rng = np.random.default_rng(seed=42)
noise = rng.normal(0, 1, 1000)
axes[1, 1].hist(noise, bins=25, color="teal", edgecolor="black", alpha=0.7)
axes[1, 1].set_title("1,000 Normal Samples (Histogram)")
axes[1, 1].grid(True)

# Adjust spacing so titles and axis labels do not overlap!
fig.tight_layout()
plt.show()
```

9.4 `plt.subplot()` for Individual Axes

You can also create subplots individually using the 3-digit shorthand `plt.subplot(rows, cols, index)`:

```
x = np.linspace(0, 10, 100)

plt.figure(figsize=(9, 4))

# Subplot 1 (Left): 1 row, 2 cols, index 1
plt.subplot(1, 2, 1)
plt.plot(x, np.sqrt(x), 'g')
plt.title("Square Root")

# Subplot 2 (Right): 1 row, 2 cols, index 2
plt.subplot(1, 2, 2)
plt.plot(x, x**0.33, 'purple')
plt.title("Cube Root")

plt.tight_layout()
plt.show()
```

Exercises and Mini-Lab: Function Family Exploration

Create a 1×3 panel figure comparing the exponential growth function $y = e^{kx}$ for three different rate constants $k \in \{0.5, 1.0, 2.0\}$:

```
x = np.linspace(0, 3, 150)
k_values = [0.5, 1.0, 2.0]
colors = ["blue", "green", "red"]

fig, axes = plt.subplots(1, 3, figsize=(12, 4), sharey=True)

for ax, k, color in zip(axes, k_values, colors):
    y = np.exp(k * x)
    ax.plot(x, y, color=color, linewidth=2)
    ax.set_title(f"k = {k}")
    ax.set_xlabel("x")
    ax.grid(True)

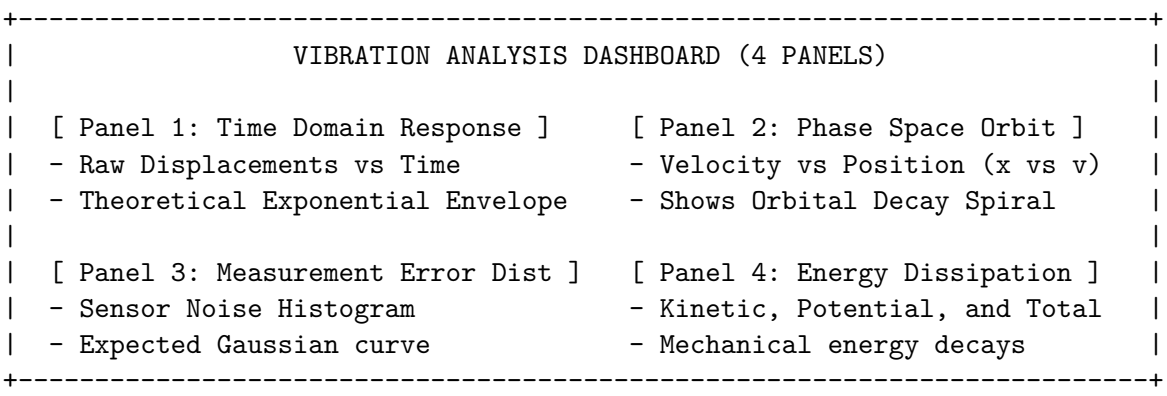
axes[0].set_ylabel("y = exp(kx)")
fig.suptitle("Exponential Growth Family", fontsize=14, fontweight="bold")
fig.tight_layout()
```

```
plt.show()
```

Milestone Project 3: Plotting Dashboard

Milestone Project 3: Scientific Plotting Dashboard

In this milestone project, you will build an end-to-end telemetry and analysis dashboard for a mechanical damping experiment. Using NumPy, you will generate simulated physical responses, compute frequency spectra, simulate measurement noise, and produce a publication-grade, 4-panel dashboard with custom styling, legends, and annotations.



Complete Project Code: `vibration_dashboard.py`

Create a file named `vibration_dashboard.py` in Spyder and run it:

```
"""
vibration_dashboard.py
======================
Milestone Project 3: 4-Panel Scientific Plotting Dashboard
for a Damped Harmonic Oscillator.
"""

import numpy as np
import matplotlib.pyplot as plt

# 1. Physics Parameters
m = 1.0         # Mass (kg)
k = 25.0        # Spring constant (N/m)
```

```
c = 0.6        # Damping coefficient (N*s/m)

# Derived natural frequency and damping ratio
omega_0 = np.sqrt(k / m)               # Natural freq = 5.0 rad/s
gamma = c / (2 * m)                    # Damping factor = 0.3
omega_d = np.sqrt(omega_0**2 - gamma**2) # Damped freq

# 2. Time Array & Trajectory Simulation
t = np.linspace(0, 15, 600)  # 15 seconds
envelope = np.exp(-gamma * t)
x_clean = envelope * np.cos(omega_d * t)
v_clean = -envelope * (gamma * np.cos(omega_d * t) + omega_d * np.sin(omega_d * t))

# Add realistic sensor noise (Gaussian)
rng = np.random.default_rng(seed=2026)
noise = rng.normal(0, 0.05, size=len(t))
x_noisy = x_clean + noise

# Energy Calculations
ke = 0.5 * m * (v_clean ** 2)
pe = 0.5 * k * (x_clean ** 2)
total_energy = ke + pe

# 3. Build the 2x2 Dashboard Figure
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Damped Harmonic Oscillator Telemetry Dashboard",
             fontsize=15, fontweight="bold", y=0.98)

# --- Panel 1: Time-Domain Position ---
ax1 = axes[0, 0]
ax1.plot(t, x_noisy, color="lightblue", alpha=0.7, label="Sensor Reading")
ax1.plot(t, x_clean, color="darkblue", linewidth=1.8, label="Analytic Model")
ax1.plot(t, envelope, "r--", linewidth=1.2, label="Decay Envelope")
ax1.plot(t, -envelope, "r--", linewidth=1.2)
ax1.set(title="Position vs Time", xlabel="Time (s)", ylabel="Displacement (m)")
ax1.grid(True, linestyle=":", alpha=0.6)
ax1.legend(loc="upper right", fontsize=9)

# --- Panel 2: Phase Space Trajectory (Velocity vs Position) ---
ax2 = axes[0, 1]
ax2.plot(x_clean, v_clean, color="purple", linewidth=1.5)
ax2.plot(x_clean[0], v_clean[0], 'go', markersize=8, label="Start (t=0)")
ax2.plot(x_clean[-1], v_clean[-1], 'ro', markersize=6, label="End (t=15s)")
ax2.set(
    title="Phase Space (Orbital Decay)",
    xlabel="Position x (m)",
```

```
    ylabel="Velocity v (m/s)",
)
ax2.grid(True, linestyle=":", alpha=0.6)
ax2.legend(loc="upper right", fontsize=9)

# --- Panel 3: Sensor Noise Distribution ---
ax3 = axes[1, 0]
counts, bins, _ = ax3.hist(
    noise, bins=25, density=True,
    color="teal", edgecolor="black", alpha=0.65,
)
# Overlay theoretical Gaussian curve
sigma = 0.05
gauss_x = np.linspace(-0.2, 0.2, 100)
gauss_y = (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(-0.5 * (gauss_x / sigma)**2)
ax3.plot(gauss_x, gauss_y, "r-", linewidth=2, label="Expected Gaussian")
ax3.set(
    title="Sensor Noise Residual Distribution",
    xlabel="Error (m)",
    ylabel="Probability Density",
)
ax3.grid(True, linestyle=":", alpha=0.6)
ax3.legend(loc="upper right", fontsize=9)

# --- Panel 4: Mechanical Energy Dissipation ---
ax4 = axes[1, 1]
ax4.plot(t, total_energy, "k-", linewidth=2.0, label="Total Energy")
ax4.plot(t, ke, "b--", linewidth=1.2, label="Kinetic (KE)")
ax4.plot(t, pe, "g:", linewidth=1.2, label="Potential (PE)")
ax4.set(
    title="Energy Dissipation Over Time",
    xlabel="Time (s)",
    ylabel="Energy (Joules)",
)
ax4.grid(True, linestyle=":", alpha=0.6)
ax4.legend(loc="upper right", fontsize=9)

# Tight layout and save figure
fig.tight_layout()
plt.savefig("vibration_dashboard.png", dpi=300, bbox_inches="tight")
print("Dashboard generated successfully and saved to 'vibration_dashboard.png'!")
plt.show()
```

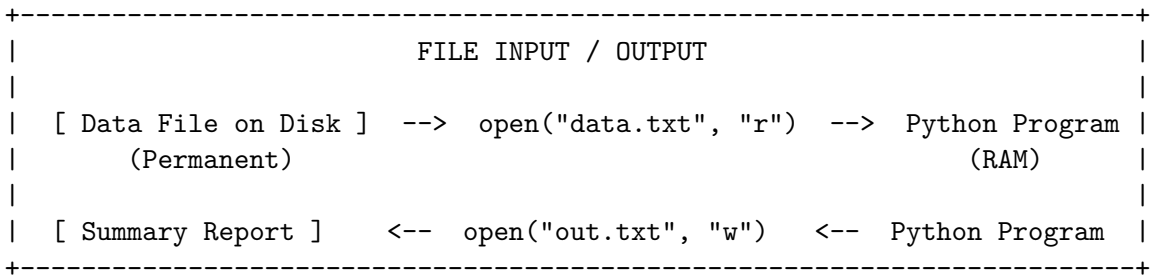
When run in Spyder, the complete multi-panel figure appears in the **Plots Pane** and a 300-DPI high-resolution PNG is saved to your project folder ready to embed in reports.

Files and Text Formatting

File Handling and I/O

Chapter 10: File Handling and I/O

Laboratory sensors, data loggers, and simulation engines store their results in files (such as `.csv`, `.txt`, or `.dat`). To analyze this data, you must be able to open files, read their contents, process numerical tables, and write processed summaries back to disk.



10.1 File Objects and `open()`

The built-in function `open(filename, mode)` connects your Python program to an operating system file and returns a **file object**:

```
file_obj = open("experiment.txt", "w")
```

10.2 File Object Attributes

File objects provide attributes to check their state:

Attribute	Description	Example
`f.name`	Name or path of the file	`'experiment.txt'`
`f.mode`	Access mode used	`'w'`
`f.closed`	True if the file has been closed	`False`

10.3 File Access Modes

The mode string tells Python what operations are permitted:

Mode	Behavior
'r'	Read-only (default). File must exist, or FileNotFoundError.
'w'	Write-only. Overwrites existing file or creates a new one!
'a'	Append-only. Adds new data to the end without deleting past.
'r+'	Read and write simultaneously.

10.4 Reading Data: `read()`, `readline()`, and `readlines()`

Python offers three ways to extract text from a file:

```
1. f.read()        --> Reads the ENTIRE file into a single giant string.
2. f.readline()    --> Reads one single line (up to next \n).
3. f.readlines()   --> Reads all lines into a LIST of strings: ['line1\n', 'line2\n']
```

Runnable Example

```python
# First, let us create a sample data file
with open("sensor_log.txt", "w") as f:
    f.write("Time(s),Voltage(V)\n")
    f.write("0.0,0.02\n")
    f.write("0.5,1.25\n")
    f.write("1.0,2.48\n")

# Method A: Iterating directly over the file line-by-line (Most memory efficient!)
print("--- Iterating line-by-line ---")
with open("sensor_log.txt", "r") as f:
    for line in f:
        # strip() removes trailing newline \n
        print("Line:", line.strip())

# Method B: readlines() to get a list
with open("sensor_log.txt", "r") as f:
```

```
    all_lines = f.readlines()
    print("\nReadlines list:", all_lines)
```

10.5 Writing Data: `write()` and `writelines()`

- **`f.write(string)`:** Writes a string. Unlike `print()`, it **does not** automatically add a newline `\n`. You must add `\n` explicitly!
- **`f.writelines(list_of_strings)`:** Writes a list of strings to the file.

Runnable Example

```
measurements = [23.1, 24.5, 23.8, 25.2]

with open("output_data.txt", "w") as f:
    f.write("# Temperature Log\n")
    for idx, val in enumerate(measurements, start=1):
        f.write(f"Sample {idx}: {val:.2f} C\n")

print("File written successfully!")
```

10.6 The `with` Statement (Context Manager)

In older Python code, you had to remember to call `f.close()` manually:

```
# Old, risky way (if an error occurs between open and close, file stays locked!)
f = open("data.txt", "r")
data = f.read()
f.close()
```

Always use the `with` statement instead:

```
with open("data.txt", "r") as f:
    data = f.read()
# The file is GUARANTEED to close automatically here, even if errors occur!
```

10.7 Reading and Writing Numerical Data with NumPy

While Python's built-in file methods work for text, NumPy provides specialized functions for reading and saving tabular numerical arrays:

- **`np.savetxt(filename, array, delimiter=",")`:** Writes an array to CSV/text.
- **`np.loadtxt(filename, delimiter=",", skiprows=1)`:** Loads text into an `ndarray`.

Runnable Example

```
import numpy as np

# 1. Generate numerical experiment data
time = np.linspace(0, 2, 5)
signal = np.sin(2 * np.pi * time)

# Combine into two columns: shape (5, 2)
dataset = np.column_stack((time, signal))

# 2. Save directly to CSV with a header
np.savetxt("signal_data.csv", dataset, delimiter=",",
           header="Time(s),Signal(V)", comments="", fmt="%.4f")
print("Saved array to 'signal_data.csv'")

# 3. Read it back in one line!
loaded_data = np.loadtxt("signal_data.csv", delimiter=",", skiprows=1)
print("\nLoaded data array back from disk:")
print(loaded_data)
print("Shape:", loaded_data.shape)
```

String Formatting

Chapter 11: String Formatting

In scientific computing, raw numbers are often printed with too many decimals (like `3.141592653589793`) or misaligned columns. Formatting strings allows you to control precision, scientific notation, field widths, and alignment so your tables and reports look clean, readable, and professional.

```
Raw Output:
Sample 1: 3.141592653589793 V
Sample 12: 125.6 V
Sample 105: 0.0004521 V

Formatted Output:
Sample   1 |   3.14 V |  3.14e+00 V
Sample  12 | 125.60 V |  1.26e+02 V
Sample 105 |   0.00 V |  4.52e-04 V
```

11.1 Modern f-strings

Introduced in modern Python, **f-strings** (formatted string literals with syntax `f"{variable:format}"`) are prefixed with an `f` or `F`. Any Python expression inside curly braces is evaluated and formatted in place:

Syntax:

```
f"...{expression:format_specifier}..."
```

Runnable Example

```
voltage = 12.45678
current = 0.045
resistance = voltage / current

print(f"Voltage:    {voltage:.2f} V")
print(f"Current:    {current:.3f} A")
print(f"Resistance: {resistance:8.1f} Ohms")
```

11.2 Format Codes, Precision, Width, and Alignment

The format specifier follows a colon `:` inside the curly braces:

```
                {value : > 10 . 3 f}
                         |  |  |  |
   Alignment (Right) -----+  |  |  +-- Type (float)
   Field Width (10 chars) ---+  +----- Precision (3 decimal places)
```

1. Type Codes:

- `s`: String (text)
- `d`: Integer
- `f`: Fixed-point floating-point number
- `e` or `E`: Scientific exponential notation (e.g., `1.23e-04`)
- `%`: Percentage (multiplies by 100 and adds `%`)

2. Alignment Options:

- `<`: Left-aligned
- `>`: Right-aligned (standard for numbers)
- `^`: Centered

Runnable Example

```python
# Demonstrating formatting types
charge = 1.60217663e-19  # Electron charge in Coulombs
count = 42
efficiency = 0.8745

print(f"Scientific notation (e): {charge:12.4e} C")
print(f"Integer padding (05d):    {count:05d}")
print(f"Percentage formatting:   {efficiency:6.2%}")

# Alignment demonstration
print("\nAlignment:")
print(f"Left:   |{'text':<10}|")
print(f"Right:  |{'text':>10}|")
print(f"Center: |{'text':^10}|")
```

11.3 Legacy % Formatting (C-Style)

You will often encounter `%` formatting in older codebases:

```
name = "Sensor-A"
reading = 4.5678

# Uses %s for string, %f for float
formatted = "Device: %s | Reading: %.2f V" % (name, reading)
print(formatted)
```

11.4 The `str.format()` Method

Introduced in Python 3.0, using `{}` placeholders:

```
name = "Sensor-A"
reading = 4.5678

formatted = "Device: {} | Reading: {:.2f} V".format(name, reading)
print(formatted)
```

11.5 Comparing the Three Approaches

```
+----------------+---------------------------------------+----------------+
| Technique      | Syntax example                        | Recommendation |
+----------------+---------------------------------------+----------------+
| f-strings      | f"{name:>10}: {val:.2f}"              | Preferred      |
| str.format()   | "{:>10}: {:.2f}".format(name, val)    | Good           |
| % operator     | "%10s: %.2f" % (name, val)            | Legacy         |
+----------------+---------------------------------------+----------------+
```

Exercises and Mini-Lab: Formatted Laboratory Data Table

Write a script that takes lists of experimental samples and prints a formatted scientific report table:

```
samples = ["Trial-1", "Trial-2", "Trial-3", "Trial-4", "Trial-5"]
frequencies = [120.45, 1250.0, 15400.8, 98.2, 542000.0]
amplitudes = [0.0452, 1.2589, 0.8540, 0.0012, 0.1250]

# Print header
print("=" * 46)
print(f"{'Sample ID':<10} | {'Frequency (Hz)':>15} | {'Amplitude (V)':>15}")
print("=" * 46)

# Print rows
for s, f, a in zip(samples, frequencies, amplitudes):
    print(f"{s:<10} | {f:15.2e} | {a:15.4f}")

print("=" * 46)
```

Output:

```
==============================================
Sample ID  |  Frequency (Hz) |   Amplitude (V)
==============================================
Trial-1    |        1.20e+02 |          0.0452
Trial-2    |        1.25e+03 |          1.2589
Trial-3    |        1.54e+04 |          0.8540
Trial-4    |        9.82e+01 |          0.0012
Trial-5    |        5.42e+05 |          0.1250
==============================================
```

Milestone Project 4: Data Logger and Report Writer

Milestone Project 4: Data Logger and Report Writer

In this milestone project, you will build a complete scientific data logging pipeline: 1. **Acquisition / Simulation:** Generate a time series of thermal cooling data according to Newton's Law of Cooling with simulated sensor noise. 2. **File Export:** Format the data into clean, aligned columns and write it to a text data log (`cooling_log.dat`) with metadata headers. 3. **File Ingestion:** Read the data file back into memory using line parsing and NumPy. 4. **Analysis & Visualization:** Compute statistical summary metrics and render a diagnostic plot with error bounds, saving the resulting graph to disk.

```
+--------------------------------------------------------------------------+
|                  DATA LOGGER AND REPORT WRITING PIPELINE                 |
|                                                                          |
|  [ Step 1: Simulate Physics ] --> T(t) = T_env + (T0 - T_env)*exp(-r*t)  |
|                                          |                               |
|  [ Step 2: Formatted Writer ] <----------+                               |
|         | Writes 'cooling_log.dat' with aligned columns & ISO metadata   |
|         v                                                                |
|  [ Step 3: Ingestion & Parser ]                                          |
|         | Reads back numerical columns into NumPy arrays                 |
|         v                                                                |
|  [ Step 4: Visual Report Plot ]                                          |
|           Produces 'cooling_report.png' with decay fit & noise envelope  |
+--------------------------------------------------------------------------+
```

Complete Project Code: `thermal_logger.py`

Create a new file in Spyder named `thermal_logger.py` and run it:

```
"""
thermal_logger.py
=================
Milestone Project 4: Generates data, writes a formatted log file,
reads it back, and plots a verification figure.
"""
```

```
import numpy as np
import matplotlib.pyplot as plt


def generate_and_log_data(filename="cooling_log.dat"):
    """
    Simulates thermal cooling: T(t) = T_env + (T0 - T_env) * exp(-r * t)
    and logs the formatted data to a file.
    """
    print("\n1. Generating thermal cooling dataset...")

    # Physics parameters
    t_env = 22.0     # Ambient temperature in Celsius
    t_0 = 95.0       # Initial temperature (boiling liquid)
    r = 0.045        # Cooling rate constant (1/s)

    # Time points from 0 to 120 seconds
    time_pts = np.linspace(0, 120, 41)  # Every 3 seconds

    # Theoretical temperature decay
    t_clean = t_env + (t_0 - t_env) * np.exp(-r * time_pts)

    # Add sensor noise (+/- 0.6 °C)
    rng = np.random.default_rng(seed=2026)
    sensor_noise = rng.normal(0.0, 0.6, size=len(time_pts))
    t_measured = t_clean + sensor_noise

    # Write formatted file
    print(f"2. Writing formatted log to '{filename}'...")
    with open(filename, "w") as f:
        # File Header and Metadata
        f.write("# ============================================================\n")
        f.write("# EXPERIMENTAL DATA LOG: Newton's Law of Cooling\n")
        f.write("# Environment: 22.0 C | Initial Temp: 95.0 C | Rate: 0.045 /s\n")
        f.write("# ============================================================\n")
        f.write(
            f"{'Index':<6} | {'Time (s)':>10} | "
            f"{'Theory (C)':>12} | {'Sensor (C)':>12}\n"
        )
        f.write("# " + "-" * 50 + "\n")

        # Data Rows
        rows = zip(time_pts, t_clean, t_measured)
        for i, (t_val, clean_val, meas_val) in enumerate(rows, start=1):
            f.write(
                f"{i:<6d} | {t_val:10.2f} | "
                f"{clean_val:12.3f} | {meas_val:12.3f}\n"
```

```
            )

    print(f">> File '{filename}' successfully written ({len(time_pts)} records).")


def read_and_plot_data(filename="cooling_log.dat"):
    """
    Reads the formatted text file back from disk and plots the results.
    """
    print(f"\n3. Reading data back from '{filename}'...")

    times = []
    theory = []
    sensor = []

    with open(filename, "r") as f:
        for line in f:
            # Skip comment and header lines starting with '#'
            if line.startswith("#") or "Time" in line or not line.strip():
                continue

            # Split row by the '|' delimiter
            parts = line.strip().split("|")
            if len(parts) == 4:
                idx = int(parts[0])
                t_val = float(parts[1])
                theo_val = float(parts[2])
                sens_val = float(parts[3])

                times.append(t_val)
                theory.append(theo_val)
                sensor.append(sens_val)

    times = np.array(times)
    theory = np.array(theory)
    sensor = np.array(sensor)

    print(f"Parsed {len(times)} records. Generating report plot...")

    # 4. Plot and Save
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 7), sharex=True,
                                   gridspec_kw={'height_ratios': [3, 1]})

    # Top Panel: Cooling Curves
    ax1.plot(times, theory, "b-", linewidth=2, label="Theoretical Model")
    ax1.scatter(
        times, sensor, color="red", marker="o", s=30,
```

```
        label="Thermocouple Sensor", zorder=3,
    )
    ax1.axhline(22.0, color="gray", linestyle="--", label="Ambient Temp (22 °C)")
    ax1.set(title="Liquid Thermal Cooling Profile", ylabel="Temperature (°C)")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend()

    # Bottom Panel: Residual Sensor Error
    residuals = sensor - theory
    ax2.axhline(0, color="black", linestyle="--", linewidth=1)
    ax2.stem(times, residuals, linefmt="k:", markerfmt="ro", basefmt="k-")
    ax2.set(
        title="Sensor Residual Errors",
        xlabel="Time (seconds)",
        ylabel="Error (C)",
        ylim=(-2.5, 2.5),
    )
    ax2.grid(True, linestyle=":", alpha=0.6)

    fig.tight_layout()
    plt.savefig("cooling_report.png", dpi=300, bbox_inches="tight")
    print(">> Report plot saved to 'cooling_report.png'!")
    plt.show()


if __name__ == "__main__":
    generate_and_log_data()
    read_and_plot_data()
```

When you run this script: 1. `cooling_log.dat` is created on disk with formatted headers and columns. 2. The file is opened, parsed, and checked for accuracy. 3. `cooling_report.png` is generated, showing the thermal decay curve and the residual sensor noise below it.

Reliable Code

Testing and Debugging

Chapter 12: Testing and Debugging

In scientific research, buggy code is dangerous. It doesn't just crash your computer—it can produce plausible-looking but subtly incorrect calculations, invalidating an entire semester of lab work.

This chapter teaches you systematic strategies for testing your functions and using Spyder's visual debugger to locate and fix bugs with confidence.

```
Bug Detection Spectrum:

[ Overt Bug ]               ------> Program crashes with a loud traceback.
(Easy to fix!)

[ Covert Bug (Silent) ]     ------> Program runs to completion, but prints
(The dangerous one!)                a wrong number (e.g. 2.45 instead of 4.90).
```

12.1 Black-Box vs Glass-Box Testing

When verifying a function, there are two distinct perspectives:

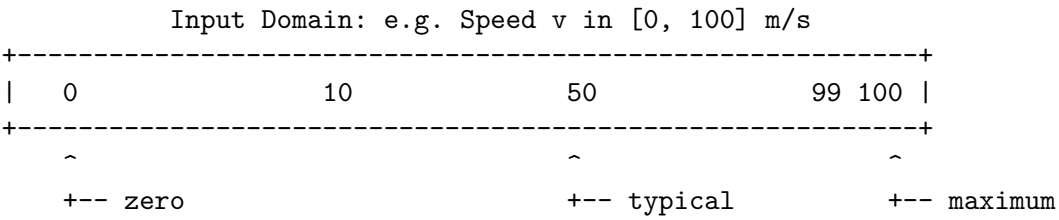
```
Black-Box Testing:
Input (X) --> [ ??? Unknown Interior ??? ] --> Output (Y)
(Test against the specification without looking at the internal code)

Glass-Box (White-Box) Testing:
Input (X) --> [ if condition -> loop -> return ] --> Output (Y)
(Design tests to exercise every internal branch, loop, and path in the code)
```

- **Black-Box Testing:** You test the function based solely on its docstring and specifications. It tests whether the software delivers what the user expects.
- **Glass-Box Testing:** You inspect the code lines directly and choose inputs specifically designed to trigger every `if`, `elif`, and `else` branch, and to test loops with zero, one, and multiple iterations.

12.2 Designing Test Cases: Typical, Edge, and Boundary Values

A good scientist does not test a function with just one random number. Always test three categories of inputs:



1. **Typical Cases:** Normal expected values (e.g., $v = 25.0$).
2. **Boundary Cases:** Values right at the limits of validity (e.g., $v = 0.0$ or $v = 100.0$).
3. **Edge / Extreme Cases:** Empty inputs (`[]`), single-element collections, negative numbers where positives are expected, or very large floats (10^{15}).

Runnable Example: Testing a Velocity Calculator

```python
def average_velocity(distance, time):
    """Computes v = d / t. Distance >= 0, Time > 0."""
    if time <= 0:
        raise ValueError("Time must be strictly positive!")
    if distance < 0:
        raise ValueError("Distance cannot be negative!")
    return distance / time

# Formal test suite
def test_average_velocity():
    # 1. Typical case
    assert average_velocity(100.0, 5.0) == 20.0, "Failed typical case"

    # 2. Boundary: Zero distance
    assert average_velocity(0.0, 5.0) == 0.0, "Failed zero distance boundary"

    # 3. Boundary: Small time step
    assert abs(average_velocity(1.0, 0.01) - 100.0) < 1e-9, "Failed small time case"

    # 4. Edge cases: Expecting exceptions
    try:
        average_velocity(50.0, 0.0)
        print("ERROR: Zero time should have raised ValueError!")
    except ValueError:
        pass  # Expected behavior!

    print("All test cases passed successfully!")
```

```
test_average_velocity()
```

12.3 Types of Bugs

Understanding the nature of a bug determines how you hunt it down:

Bug Category	Characteristics	How to Catch It
Overt	Obvious crash with an explicit Python error (e.g. `ZeroDivisionError`).	Read the traceback line number.
Covert (Silent)	The code runs without error but outputs incorrect physics numbers.	Unit tests and assertion checks.
Persistent	Occurs every single time you run the script with the same inputs.	Reproduce it in Spyder with a breakpoint.
Intermittent	Occurs unpredictably (often caused by unseeded random numbers or file race conditions).	Fix random seeds (`seed=42`) to make it persistent.

12.4 Hypothesis-and-Confirmation Debugging

Do not change lines of code at random hoping the bug disappears! Follow the scientific method:

1. `Observe the failure: "Output is negative (-5.4), but energy must be >= 0."`
2. `Formulate a hypothesis: "Perhaps velocity was subtracted instead of added."`
3. `Design an experiment: Insert a print statement or check the Variable Explorer.`
4. `Confirm or reject: Check whether the intermediate variable matches expectation.`
5. `Apply the fix: Test again to verify the bug is eliminated.`

12.5 IDE-Based Debugging in Spyder

Spyder provides a world-class graphical debugger built directly into the Editor and Console.

```
Editor Margin:
Line 14:   [*]  result = calculate_force(m, a)  <-- Red Breakpoint Dot!
```

Debugging Steps:

1. **Set a Breakpoint:** Click in the Editor margin next to any line number. A **red dot (*)** appears.
2. **Start Debugger:** Press `Ctrl + F5` (or click the blue Pause/Play debug icon).
3. The program runs until it reaches your breakpoint, then pauses execution!
4. The IPython console prompt changes to `ipdb>`.
5. **Inspect State:** Look at the **Variable Explorer**—you can see the exact values of all variables at this exact split-second in time!
6. **Step Through:**
   - **Step Over (`Ctrl + F10`):** Execute the current line and advance to the next.
   - **Step Into (`Ctrl + F11`):** Dive inside a function call to see how it computes.
   - **Continue (`Ctrl + F12`):** Resume full-speed execution until the next breakpoint.
   - **Stop (`Ctrl + Shift + F12`, or `Cmd + Shift + F12` on macOS):** Quit the debugging session. `Shift + F12` sets a conditional breakpoint. It does not stop the debugger.

12.6 Debugging a Recursive Function Step by Step

Let us debug a recursive countdown function to see the call stack in action:

```python
def countdown(n):
    print(f"Entering countdown(n={n})")
    if n == 0:
        print("Blast off! (Base case reached)")
        return
    countdown(n - 1)
    print(f"Exiting countdown(n={n})")

countdown(3)
```

Output Trace:

```
Entering countdown(n=3)
Entering countdown(n=2)
Entering countdown(n=1)
```

```
Entering countdown(n=0)
Blast off! (Base case reached)
Exiting countdown(n=1)
Exiting countdown(n=2)
Exiting countdown(n=3)
```

Notice how the frames unwind in reverse order from the stack! If you omit the base case `if n == 0`, Python recursively calls until hitting `RecursionError: maximum recursion depth exceeded`.

Exceptions

Chapter 13: Exceptions

When an unexpected event occurs during program execution—such as trying to divide by zero, reading a missing file, or parsing text where a number was expected—Python creates an **Exception**.

If your program does not handle the exception, it halts immediately with an error message (crashes). With **exception handling**, your programs can anticipate errors, recover gracefully, and continue executing.

```
Normal Path:
try:  --> [ Do critical calculation ] --> Success!

Exception Path:
try:  --> [ Division by zero occurs! ]
                |
                v
except: --> [ Catch error, log warning, return fallback value ]
```

13.1 Common Exceptions in Scientific Computing

Here are the six most common exceptions you will encounter:

Exception Type	Common Cause in Science Code	Example
`ZeroDivisionError`	Dividing by 0 in physics formulas	`1.0 / 0.0`
`ValueError`	Correct type, but invalid value	`float("apple")` or `math.sqrt(-1.0)`
`TypeError`	Operation applied to incompatible types	`"Voltage: " + 5.0` (str + float)
`IndexError`	Trying to access an index outside list bounds	`data = [1, 2]; val = data[5]`
`NameError`	Misspelling a variable or function name	`pritn("hello")`
`FileNotFoundError`	Attempting to open a data file that does not exist	`open("missing_lab.csv", "r")`

13.2 The `try–except` Block

The fundamental construct for handling errors:

```
try:
    # Risky code that might fail
    resistance = float(input("Enter resistance (Ohms): "))
    current = 12.0 / resistance
    print(f"Current is: {current:.2f} A")
except:
    print("An error occurred during calculation!")
```

> **College Tip:** Avoid bare `except:`. It catches everything, including user interruptions (`Ctrl+C`) and typos! Always catch **specific** exceptions.

13.3 Handling Specific Exceptions & Multiple Clauses

You can chain multiple `except` blocks to handle different failure modes appropriately:

Runnable Example

```
def safe_sensor_parser(raw_string, default_val=0.0):
    """Parses a numerical sensor string, returning default_val if corrupted."""
    try:
        clean_text = raw_string.strip()
        reading = float(clean_text)
        return reading
    except ValueError:
        print(
            f"Warning: Corrupted text '{raw_string}' "
            "could not be converted to float."
        )
        return default_val
    except AttributeError:
        print(
            "Warning: Expected string input, but received "
            f"type {type(raw_string)}."
        )
        return default_val

# Testing different inputs
print("Test 1 (Valid):    ", safe_sensor_parser("  24.85  "))
```

```
print("Test 2 (Corrupt):  ", safe_sensor_parser("N/A_SENSOR_TIMEOUT"))
print("Test 3 (Wrong type):", safe_sensor_parser(None))
```

13.4 `else` and `finally` Clauses

The complete structure includes `else` and `finally`:

```
try:
    # Risky action (e.g. open file, read data)
except FileNotFoundError:
    # Executes ONLY if FileNotFoundError happens
else:
    # Executes ONLY if NO exception was raised!
finally:
    # ALWAYS executes, whether an error occurred or not!
    # (Ideal for closing hardware connections or cleanup)
```

Runnable Example

```
def process_data_file(filename):
    file = None
    try:
        print(f"\nAttempting to open '{filename}'...")
        file = open(filename, "r")
        content = file.read()
    except FileNotFoundError:
        print(f"Error: File '{filename}' was not found on disk.")
    else:
        print("Success! File read without issues.")
        print(f"Total characters: {len(content)}")
    finally:
        if file is not None and not file.closed:
            file.close()
            print("Cleanup: File object closed safely.")
        print("Done with file operation attempt.")

# Test with non-existent file
process_data_file("non_existent_lab_reading.txt")
```

13.5 Raising Exceptions with `raise`

When writing your own scientific library, you should actively validate parameters and `raise` meaningful errors if arguments violate physical laws:

```
import math

def circle_area(radius):
    """Compute area of a circle. Radius must be non-negative."""
    if not isinstance(radius, (int, float)):
        raise TypeError(f"Radius must be numeric, got {type(radius).__name__}")
    if radius < 0:
        raise ValueError(f"Radius cannot be negative! Received: {radius}")
    return math.pi * (radius ** 2)

# Calling with invalid argument
try:
    area = circle_area(-5.0)
except ValueError as e:
    print("Caught validation error:", e)
```

13.6 Passing Exceptions Up the Call Stack

If a function does not handle an exception, the exception automatically bubbles up to the calling function. This allows low-level helper functions to signal errors while letting high-level coordinator functions decide how to handle them.

```
def read_raw_value():
    return float("invalid_data")  # Raises ValueError

def experiment_runner():
    # Catches the error that originated inside read_raw_value()
    try:
        val = read_raw_value()
    except ValueError:
        print("experiment_runner: Handled bad sensor reading gracefully!")

experiment_runner()
```

Exercises and Mini-Lab: Safe Experimental Calculator

Create a robust command-line calculator that asks the user for two numbers and an operator (`+`, `-`, `*`, `/`), handling all potential errors cleanly:

```
def safe_calculator(num1_str, num2_str, op):
    try:
        a = float(num1_str)
        b = float(num2_str)

        if op == "+":
            return a + b
        elif op == "-":
            return a - b
        elif op == "*":
            return a * b
        elif op == "/":
            return a / b
        else:
            raise ValueError(f"Unknown operator: '{op}'")

    except ValueError as val_err:
        return f"Input Error: {val_err}"
    except ZeroDivisionError:
        return "Math Error: Cannot divide by zero!"

# Test cases
print("10 / 2:   ", safe_calculator("10", "2", "/"))      # 5.0
print("10 / 0:   ", safe_calculator("10", "0", "/"))      # Math Error
print("Bad text: ", safe_calculator("ten", "2", "+"))     # Input Error
print("Bad op:   ", safe_calculator("5", "3", "^"))       # Input Error
```

Capstone Project: Robust Data Analysis Pipeline

Capstone Project: Robust Data Analysis Pipeline

Congratulations on making it to the Capstone Project! In this comprehensive project, you will unite everything you have learned throughout this book into an industrial-grade, robust scientific data pipeline.

```
+--------------------------------------------------------------------------+
|                  CAPSTONE: ROBUST DATA ANALYSIS PIPELINE                 |
|                                                                          |
|  [ 1. Ingestion Engine ]                                                 |
|     - Reads 'spectrometry_raw.csv' with corrupted entries & comments     |
|     - Uses try-except guards to filter bad data without crashing         |
|                                                                          |
|  [ 2. Numerical Processing (NumPy) ]                                     |
|     - Subtracts a baseline fitted on the spectrum edges                  |
|     - Finds the tallest peak and integrates the band                     |
|                                                                          |
|  [ 3. Visualization Dashboard (Matplotlib) ]                             |
|     - 3-panel figure: Raw vs Filtered, Peak Localization, Residuals      |
|     - Exports publication-ready 'spectrometry_analysis.png' (300 DPI)    |
|                                                                          |
|  [ 4. Executive Summary Report Writer ]                                  |
|     - Generates 'spectrometry_report.txt' with f-string formatted tables |
+--------------------------------------------------------------------------+
```

---

1. Pipeline Architecture

The pipeline consists of four modular functions designed for clean separation of concerns: 1. `generate_mock_laboratory_file(filename)` — Generates a realistic CSV file with headers, comments, valid sensor data, and corrupted values. 2. `load_and_clean_data(filename)` — Opens the file, applies defensive exception handling to discard corrupted rows, and returns clean NumPy arrays. 3. `analyze_spectrum(wavelengths, intensities)` — Performs numerical baseline removal, smooths signal noise, locates emission peaks, and calculates total energy. 4. `generate_visual_and_text_report(...)` — Renders the multi-panel figure and writes a formatted executive summary file.

---

2. Complete Pipeline Implementation: `pipeline_capstone.py`

Create a file named `pipeline_capstone.py` in your Spyder editor and run it (F5):

```
"""
pipeline_capstone.py
====================
Capstone Project: An end-to-end robust scientific analysis pipeline
incorporating File I/O, Exception Handling, NumPy, and Matplotlib.
"""

import os
import numpy as np
import matplotlib.pyplot as plt


def generate_mock_laboratory_file(filename="spectrometry_raw.csv"):
    """Generates mock optical emission spectrometry data with corrupted rows."""
    print(f"[Stage 1] Generating raw laboratory file: '{filename}'...")

    # 200 wavelength points from 400 nm to 700 nm (visible light spectrum)
    wavelengths = np.linspace(400.0, 700.0, 201)

    # Emission peaks at 480nm (Hydrogen-beta) and 656nm (Hydrogen-alpha)
    peak1 = 35.0 * np.exp(-0.5 * ((wavelengths - 486.1) / 8.0)**2)
    peak2 = 80.0 * np.exp(-0.5 * ((wavelengths - 656.3) / 10.0)**2)
    baseline = 15.0 + 0.02 * wavelengths

    rng = np.random.default_rng(seed=2026)
    noise = rng.normal(0, 1.5, size=len(wavelengths))
    intensity = baseline + peak1 + peak2 + noise

    with open(filename, "w") as f:
        f.write("# OPTICAL EMISSION SPECTROMETER LAB RUN 42\n")
        f.write("# Instrument: OceanOptics-HR2000 | Operator: College Lab Team\n")
        f.write("Wavelength(nm),Intensity(counts)\n")

        for i, (w, intens) in enumerate(zip(wavelengths, intensity)):
            # Introduce intentional realistic data corruption at isolated indices
            if i == 45:
                f.write("467.50,SENSOR_COMM_TIMEOUT\n")  # Corrupted float
            elif i == 110:
                f.write("CORRUPTED_LINE_DROPPED_SYNC\n")   # Missing column
            elif i == 160:
                f.write("640.00,-9999.00\n")             # Out-of-bounds error code
            else:
                f.write(f"{w:.2f},{intens:.3f}\n")
```

```
    print(f"         Generated {len(wavelengths)} rows with 3 intentional flaws.")


def load_and_clean_data(filename="spectrometry_raw.csv"):
    """
    Reads CSV file with defensive exception handling, skipping bad records.
    """
    print(f"[Stage 2] Ingesting and sanitizing data from '{filename}'...")

    clean_w = []
    clean_i = []
    dropped_count = 0

    if not os.path.exists(filename):
        raise FileNotFoundError(f"Required data file '{filename}' was not found.")

    with open(filename, "r") as f:
        for line_num, line in enumerate(f, start=1):
            line_str = line.strip()

            # Skip comments and header
            if line_str.startswith("#") or "Wavelength" in line_str or not line_str:
                continue

            parts = line_str.split(",")
            try:
                if len(parts) != 2:
                    raise IndexError(f"Expected 2 columns, found {len(parts)}")

                w_val = float(parts[0])
                i_val = float(parts[1])

                # Check physical validity (negative readings indicate hardware error)
                if i_val < 0 or w_val <= 0:
                    raise ValueError(
                        f"Physically invalid measurement: ({w_val}, {i_val})"
                    )

                clean_w.append(w_val)
                clean_i.append(i_val)

            except (ValueError, IndexError) as err:
                dropped_count += 1
                # In production code, log the error rather than crashing
                # print(f"         Skipping row {line_num}: {err}")
```

```
    print(
        f"         Successfully parsed {len(clean_w)} valid rows "
        f"(Filtered out {dropped_count} bad rows)."
    )
    return np.array(clean_w), np.array(clean_i)


def analyze_spectrum(wavelengths, intensities):
    """Computes spectral features, peak wavelengths, and background."""
    print("[Stage 3] Running NumPy numerical processing...")

    # 1. Baseline estimation using linear fit on edges
    left_mask = wavelengths < 430.0
    right_mask = wavelengths > 680.0
    edge_w = np.concatenate([wavelengths[left_mask], wavelengths[right_mask]])
    edge_i = np.concatenate([intensities[left_mask], intensities[right_mask]])

    # Fit line: y = m*x + b
    slope, intercept = np.polyfit(edge_w, edge_i, deg=1)
    baseline = slope * wavelengths + intercept

    # 2. Baseline subtraction
    corrected_intensity = intensities - baseline

    # 3. Peak Identification
    peak_idx = np.argmax(corrected_intensity)
    peak_w = wavelengths[peak_idx]
    peak_max_counts = corrected_intensity[peak_idx]

    # 4. Integrated Signal Area (Trapezoidal Rule)
    total_area = np.trapezoid(np.maximum(corrected_intensity, 0), wavelengths)

    results = {
        "baseline": baseline,
        "corrected": corrected_intensity,
        "peak_wavelength": peak_w,
        "peak_intensity": peak_max_counts,
        "integrated_area": total_area,
        "mean_noise_std": np.std(edge_i - (slope * edge_w + intercept))
    }
    return results


def generate_visual_and_text_report(w, i_raw, analysis,
                                    fig_name="spectrometry_analysis.png",
                                    report_name="spectrometry_report.txt"):
```

```
    """Creates a publication plot and formats an executive summary."""
    print("[Stage 4] Producing output visualizations and executive report...")

    # --- Part A: 3-Panel Matplotlib Figure ---
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(9, 10), sharex=True)
    fig.suptitle(
        "Spectrometry Acquisition & Diagnostic Analysis",
        fontsize=14, fontweight="bold",
    )

    # Subplot 1: Raw Signal and Baseline
    ax1.plot(w, i_raw, "b.", markersize=3, label="Raw Sensor Readings")
    ax1.plot(
        w, analysis["baseline"], "r--", linewidth=1.8,
        label="Estimated Linear Baseline",
    )
    ax1.set(ylabel="Intensity (counts)", title="Raw Spectral Profile & Baseline")
    ax1.legend(loc="upper left")
    ax1.grid(True, linestyle=":", alpha=0.6)

    # Subplot 2: Corrected Signal & Peak Marker
    corr = analysis["corrected"]
    ax2.plot(w, corr, "darkgreen", linewidth=1.5, label="Baseline-Corrected Signal")
    ax2.plot(
        analysis["peak_wavelength"],
        analysis["peak_intensity"],
        "r*",
        markersize=14,
        label=f"Dominant Peak: {analysis['peak_wavelength']:.1f} nm",
    )
    ax2.fill_between(
        w, 0, np.maximum(corr, 0),
        color="green", alpha=0.15, label="Integrated Band",
    )
    ax2.set(ylabel="Corrected (counts)", title="Baseline-Corrected Emission Peaks")
    ax2.legend(loc="upper left")
    ax2.grid(True, linestyle=":", alpha=0.6)

    # Subplot 3: Residual Edge Noise
    residuals = i_raw - analysis["baseline"]
    ax3.axhline(0, color="black", linestyle="--")
    ax3.plot(w, residuals, color="gray", linewidth=1.0, label="Noise Residuals")
    ax3.set(xlabel="Wavelength (nm)", ylabel="Residuals", title="Sensor Noise Floor")
    ax3.legend(loc="upper left")
    ax3.grid(True, linestyle=":", alpha=0.6)

    fig.tight_layout()
```

```
    plt.savefig(fig_name, dpi=300, bbox_inches="tight")
    print(f"         Saved figure: '{fig_name}'")
    plt.show()

    # --- Part B: Executive Text Summary ---
    with open(report_name, "w") as f:
        f.write("=" * 65 + "\n")
        f.write("      EXECUTIVE SPECTROMETRY ANALYSIS SUMMARY REPORT\n")
        f.write("=" * 65 + "\n")
        f.write(f"Wavelength Range Monitored:   [{w[0]:.1f} nm - {w[-1]:.1f} nm]\n")
        f.write(f"Valid Samples Processed:      {len(w):d}\n")
        f.write(
            "Estimated Noise Floor (Std):  "
            f"{analysis['mean_noise_std']:.3f} counts\n"
        )
        f.write("-" * 65 + "\n")
        f.write("IDENTIFIED SPECTRAL FEATURES:\n")
        f.write(
            "  * Dominant Emission Peak:   "
            f"{analysis['peak_wavelength']:8.2f} nm\n"
        )
        f.write(
            "  * Peak Signal Height:       "
            f"{analysis['peak_intensity']:8.2f} counts\n"
        )
        f.write(
            "  * Total Integrated Power:   "
            f"{analysis['integrated_area']:8.2f} counts*nm\n"
        )
        f.write("=" * 65 + "\n")
        f.write("STATUS: Pipeline executed with ZERO unhandled exceptions.\n")

    print(f"         Saved summary: '{report_name}'")


if __name__ == "__main__":
    # Execute full pipeline end-to-end
    raw_file = "spectrometry_raw.csv"
    generate_mock_laboratory_file(raw_file)
    w_clean, i_clean = load_and_clean_data(raw_file)
    spectral_metrics = analyze_spectrum(w_clean, i_clean)
    generate_visual_and_text_report(w_clean, i_clean, spectral_metrics)
    print("\n[SUCCESS] Capstone analysis pipeline completed successfully!")
```

Console Output:

```
[Stage 1] Generating raw laboratory file: 'spectrometry_raw.csv'...
          Generated 201 rows with 3 intentional flaws.
[Stage 2] Ingesting and sanitizing data from 'spectrometry_raw.csv'...
          Successfully parsed 198 valid rows (Filtered out 3 bad rows).
[Stage 3] Running NumPy numerical processing...
[Stage 4] Producing output visualizations and executive report...
          Saved figure: 'spectrometry_analysis.png'
          Saved summary: 'spectrometry_report.txt'

[SUCCESS] Capstone analysis pipeline completed successfully!
```

You now possess a complete template for reading real experimental files, guarding against sensor drops, computing statistics with NumPy, plotting multi-panel figures with Matplotlib, and exporting formatted executive reports!

Appendices

Appendix A: Spyder Shortcuts Cheat Sheet

Appendix A: Spyder Keyboard Shortcuts Cheat Sheet

Mastering keyboard shortcuts in Spyder 6 makes your coding workflow much faster. Here are the essential shortcuts for Windows, Linux, and macOS:

Action	Windows / Linux	macOS	Description
Run entire file	`F5`	`F5`	Executes the active script from line 1 to the end.
Run current cell	`Ctrl + Enter`	`Cmd + Enter`	Runs only the block enclosed by `#%%`.
Run cell & advance	`Shift + Enter`	`Shift + Enter`	Runs the cell and moves cursor to next cell.
Run selection	`F9`	`F9`	Executes only highlighted lines of code.
Switch to Console	`Ctrl + Shift + I`	`Cmd + Shift + I`	Moves focus to the IPython console.
Switch to Editor	`Ctrl + Shift + E`	`Cmd + Shift + E`	Moves focus back to code editor.
Inspect Documentation	`Ctrl + I`	`Cmd + I`	Opens function docstring in Help pane.
Comment / Uncomment	`Ctrl + 1`	`Cmd + 1`	Toggles `#` on selected lines.
Indent line(s)	`Tab`	`Tab`	Shifts code 4 spaces right.
Unindent line(s)	`Shift + Tab`	`Shift + Tab`	Shifts code 4 spaces left.
Find and Replace	`Ctrl + F` / `Ctrl + R`	`Cmd + F` / `Cmd + R`	Search and replace text.
Set / Clear Breakpoint	`F12`	`F12`	Toggles a breakpoint. `Shift + F12` sets a condition.
Start Debugging	`Ctrl + F5`	`Cmd + F5`	Starts execution in debug mode.
Step Over	`Ctrl + F10`	`Cmd + F10`	Executes current line and pauses at next.

Action	Windows / Linux	macOS	Description
Step Into	`Ctrl + F11`	`Cmd + F11`	Dives inside function call.
Continue Debugging	`Ctrl + F12`	`Cmd + F12`	Resumes execution until next breakpoint.
Stop Debugging	`Ctrl + Shift + F12`	`Cmd + Shift + F12`	Exits debug mode.
Clear Console	`Ctrl + L`	`Cmd + K`	Clears text from IPython prompt.

Appendix B: NumPy Quick Reference

Appendix B: NumPy Quick Reference

A practical desktop reference for everyday NumPy array operations in scientific computing.

1. Array Creation

Command	Description	Example
`np.array(list)`	Create array from list	`np.array([1, 2, 3])`
`np.zeros(shape)`	Array filled with zeros	`np.zeros((3, 3))`
`np.ones(shape)`	Array filled with ones	`np.ones(5)`
`np.full(shape, val)`	Array with constant value	`np.full((2, 2), 9.8)`
`np.arange(start, stop, step)`	Sequence with defined step	`np.arange(0, 10, 2)`
`np.linspace(start, stop, num)`	Evenly spaced points	`np.linspace(0, 1, 101)`
`np.eye(n)`	$n \times n$ Identity matrix	`np.eye(4)`

2. Array Inspection and Reshaping

Attribute / Method	Description
`arr.shape`	Tuple of array dimensions `(rows, cols)`
`arr.ndim`	Number of dimensions (1 for vector, 2 for matrix)
`arr.size`	Total number of elements
`arr.dtype`	Data type of elements (`float64`, `int32`, etc.)
`arr.reshape(rows, cols)`	Reshape into new dimensions
`arr.flatten()`	Flatten multi-dimensional array into 1D
`arr.T`	Transpose matrix (swap rows and columns)

3. Mathematical and Statistical Reductions

Function	Description
`arr.sum(axis=None)`	Sum of all elements (or along `axis=0` / `axis=1`)
`arr.mean(axis=None)`	Arithmetic mean
`arr.std(axis=None)`	Standard deviation

Function	Description
`arr.var(axis=None)`	Variance
`arr.min()`, `arr.max()`	Minimum and maximum values
`arr.argmin()`, `arr.argmax()`	Indices of min and max elements
`np.cumsum(arr)`	Cumulative sum array

4. Stacking and Joining

Function	Description
`np.vstack([A, B])`	Stack arrays vertically (row-wise)
`np.hstack([A, B])`	Stack arrays horizontally (column-wise)
`np.column_stack((x, y))`	Combine 1D vectors into 2D table columns

5. Linear Algebra (`np.linalg`)

Function	Mathematical Operation
`np.linalg.det(A)`	Determinant $\det(A)$
`np.linalg.inv(A)`	Matrix Inverse A^{-1}
`np.linalg.solve(A, b)`	Solve linear system $A\vec{x} = \vec{b}$
`np.linalg.eig(A)`	Eigenvalues and eigenvectors
`A @ B` or `np.matmul(A, B)`	Matrix multiplication
`np.dot(u, v)`	Vector dot product

Appendix C: Matplotlib Properties Reference

Appendix C: Matplotlib Property Reference

A visual and property quick-guide for styling scientific plots in Matplotlib.

1. Line Styles (`linestyle` or `ls`)

Specifier	Name	Visual Appearance
`'-'` or `'solid'`	Solid line	`--------------`
`'--'` or `'dashed'`	Dashed line	`--- --- --- ---`
`':'` or `'dotted'`	Dotted line	`· · · · · · · · ·`
`'-.'` or `'dashdot'`	Dash-dot line	`--- · --- · ---`
`'none'` or `''`	No line (scatter only)	

2. Marker Styles (`marker`)

Specifier	Marker Symbol	Description
`'o'`	●	Circle
`'s'`	■	Square
`'^'`		Triangle Up
`'v'`		Triangle Down
`'d'` or `'D'`	♦	Diamond
`'*'`	⋆	Star
`'+'`	+	Plus sign
`'x'`	×	Cross
`'.'`	·	Point

3. Common Color Codes (`color` or `c`)

Code	Color Name	Hex Code Equivalent
`'b'`	Blue	`#0000FF`
`'g'`	Green	`#008000`
`'r'`	Red	`#FF0000`
`'c'`	Cyan	`#00BFBF`
`'m'`	Magenta	`#BF00BF`

Code	Color Name	Hex Code Equivalent
`'y'`	Yellow	`#BFBF00`
`'k'`	Black	`#000000`
`'w'`	White	`#FFFFFF`
`'orange'`	Scientific Orange	`#FF7F0E`
`'teal'`	Teal	`#008080`

4. Key Plot Method Arguments

```
plt.plot(x, y,
         color='darkblue',       # Color of line
         linestyle='--',         # Dash pattern
         linewidth=2.0,          # Thickness of line in points
         marker='o',             # Shape of points
         markersize=6,           # Point diameter
         markerfacecolor='red',  # Fill color inside marker
         markeredgecolor='black',# Border outline of marker
         alpha=0.8,              # Transparency: 0.0 (clear) to 1.0 (opaque)
         label='Run 1')          # Legend text
```

Appendix D: f-string Format Code Reference

Appendix D: f-string Format Code Reference

Syntax:

```
f"{value:[fill][align][width][,_][.precision][type]}"
```

1. Type Conversion Specifiers

Code	Meaning	Input Example	Specifier	Output String
`d`	Decimal Integer	`42`	`{42:4d}`	`'  42'`
`f`	Fixed-point Float	`3.14159`	`{3.14159:.2f}`	`'3.14'`
`e`	Scientific Exponent (lower)	`0.000452`	`{0.000452:.2e}`	`'4.52e-04'`
`E`	Scientific Exponent (upper)	`0.000452`	`{0.000452:.2E}`	`'4.52E-04'`
`%`	Percentage	`0.852`	`{0.852:.1%}`	`'85.2%'`
`s`	String	`'data'`	`{'data':>8s}`	`'    data'`
`b`	Binary	`10`	`{10:08b}`	`'00001010'`
`x`	Hexadecimal	`255`	`{255:02x}`	`'ff'`

2. Width, Alignment, and Padding

Format Code	Effect	Output
`{val:>10}`	Right-aligned in 10-char space	`'      3.14'`
`{val:<10}`	Left-aligned in 10-char space	`'3.14      '`
`{val:^10}`	Centered in 10-char space	`'   3.14   '`
`{count:05d}`	Zero-padded 5-digit integer	`'00042'`
`{num:12,d}`	Comma thousand separator	`'   1,000,000'`
`{1000000:_.2f}`	Underscore thousands separator	`'1_000_000.00'`

3. Useful Debugging Shorthand (Python 3.8+)

Appending `=` prints the variable name along with its evaluated value:

```
x = 25.4
y = 10.2
print(f"{x=}, {y=}, {x*y=:.2f}")
# Output: x=25.4, y=10.2, x*y=259.08
```

Appendix E: Common Error Messages and Fixes

Appendix E: Common Error Messages and Fixes

When Python throws an error, don't panic! The traceback tells you the exact line number and nature of the issue. Here are the most frequent errors college students encounter and how to fix them:

1. `IndexError: list index out of range`

- **What happened:** You tried to access an element at an index that doesn't exist (e.g., accessing index 5 in a 3-element list).
- **Remember:** In Python, indices run from `0` to `len(list) - 1`.
- **Fix:** Check `len(my_list)` before indexing, or use `for item in my_list:` instead of index loops.

2. `TypeError: can only concatenate str (not "float") to str`

- **What happened:** You tried to add a number to text with `+` (e.g. `"Temp: " + 23.5`).
- **Fix:** Use an f-string: `f"Temp: {23.5}"` or convert with `str(23.5)`.

3. `ValueError: could not convert string to float: '...'`

- **What happened:** You passed non-numeric text to `float()` (e.g. `float("N/A")` or an unstripped header line from a file).
- **Fix:** Clean and strip the string, or wrap the conversion in a `try...except ValueError` block.

4. `NameError: name 'np' is not defined`

- **What happened:** You used a variable or imported library before defining or importing it.
- **Fix:** Ensure `import numpy as np` is at the very top of your file.

5. `IndentationError: expected an indented block`

- **What happened:** Python requires 4 spaces for indented blocks after `def`, `for`, `if`, or `try:`.
- **Fix:** Select the affected lines in Spyder and press `Tab` (or `Shift + Tab` to unindent). Avoid mixing tabs and spaces.

6. `ValueError: operands could not be broadcast together with shapes (4,) (3,)`

- **What happened:** In NumPy, you tried to perform an elementwise operation (like `a + b` or `a * b`) on two arrays with incompatible shapes.
- **Fix:** Check `a.shape` and `b.shape` in Spyder's Variable Explorer to ensure their dimensions align.

7. `ModuleNotFoundError: No module named '...'`

- **What happened:** Python cannot find the `.py` file you tried to import.
- **Fix:** In Spyder, check your working directory (top-right folder path) and ensure it points to the folder containing your module.

Appendix F: Managing Packages with Conda and Pip

Appendix F: Managing Packages with Conda and Pip

While Anaconda comes pre-installed with over 200 scientific packages, you may eventually need to install specialized libraries (such as `scikit-learn`, `sympy`, or `pandas`).

1. Conda vs Pip

- **`conda`:** The package manager built into Anaconda. It handles both Python packages and low-level C/Fortran binary dependencies (essential for scientific libraries).
- **`pip`:** The standard Python package installer. Use `pip` only if a package is unavailable via `conda`.

2. Essential Conda Commands

Run these commands in your **Anaconda Prompt** (Windows) or **Terminal** (macOS/Linux):

```
# 1. Search for a package
conda search scipy

# 2. Install a package in your active environment
conda install scipy -y

# 3. Install a package from conda-forge (community channel)
conda install -c conda-forge sympy -y

# 4. List all installed packages and their versions
conda list

# 5. Update a package to the latest release
conda update numpy -y

# 6. Remove an unwanted package
conda remove package_name
```

3. Environment Management Workflow

```
# Create a new clean environment named 'physics_lab'
conda create -n physics_lab -c conda-forge \
    python=3.12 numpy matplotlib spyder -y

# Activate it
conda activate physics_lab

# Export your environment to a shareable YAML file for lab partners
conda env export > environment.yml

# Re-create an environment from someone else's YAML file
conda env create -f environment.yml
```

Appendix G: Suggested Reading

Appendix G: Suggested Reading

To continue expanding your computational skills beyond this introductory book, here are highly recommended resources:

1. Introductory Programming

- **Python Crash Course (3rd Edition, Eric Matthes, No Starch Press):** A hands-on, project-based introduction to general-purpose Python.
- **Think Python: How to Think Like a Computer Scientist (3rd Edition, Allen B. Downey, O'Reilly):** An exceptional conceptual guide that teaches computer science thinking and algorithm design.

2. Scientific Python Ecosystem

- **Official NumPy Documentation & User Guide (`https://numpy.org/doc/`):** Comprehensive tutorials on broadcasting, indexing, and vectorization techniques.
- **Official Matplotlib Documentation & Gallery (`https://matplotlib.org/stable/gallery/`):** The gallery contains hundreds of visual examples with source code that you can adapt for your own research papers.
- **Python Data Science Handbook (Jake VanderPlas, O'Reilly):** Detailed coverage of NumPy, pandas, Matplotlib, and Scikit-Learn.
- **SciPy Lecture Notes (`https://scipy-lectures.org/`):** Outstanding self-paced tutorials for scientists and engineers.

Appendix H: Solutions to Selected Exercises

Appendix H: Solutions to Selected Exercises

Here are reference solutions and explanations for selected exercises and mini-labs from across the book.

Chapter 1: Lists

Exercise 1: Row and Column Sums

```
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]

# Calculate row sums using list comprehension
row_sums = [sum(row) for row in matrix]
print("Row sums:", row_sums)  # [60, 150, 240]

# Calculate column sums
num_cols = len(matrix[0])
col_sums = [
    sum(matrix[r][c] for r in range(len(matrix)))
    for c in range(num_cols)
]
print("Column sums:", col_sums)  # [120, 150, 180]
```

Chapter 2: Functions

Exercise 1: Recursive vs Iterative Fibonacci

```
def fib_recursive(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    return fib_recursive(n - 1) + fib_recursive(n - 2)

def fib_iterative(n):
    if n <= 0:
        return 0
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

print("10th Fibonacci (recursive):", fib_recursive(10))  # 55
print("10th Fibonacci (iterative):", fib_iterative(10))  # 55
```

Chapter 7: Linear Algebra

Circuit Mesh Analysis

```
import numpy as np

# R @ I = V
R = np.array([
    [12.0, -4.0],
    [-4.0, 10.0]
])

V = np.array([18.0, 0.0])

# Solve for currents
I = np.linalg.solve(R, V)
print(f"Current I1 = {I[0]:.3f} A, Current I2 = {I[1]:.3f} A")
# I1 = 1.731 A, I2 = 0.692 A
```

Chapter 13: Exceptions

Safe Calculator

```
def safe_calculator(num1_str, num2_str, op):
    try:
        a = float(num1_str)
        b = float(num2_str)
        if op == "+":
            return a + b
        elif op == "-":
            return a - b
        elif op == "*":
            return a * b
        elif op == "/":
            return a / b
        else:
            raise ValueError(f"Unknown operator: '{op}'")
    except ValueError as e:
        return f"Input Error: {e}"
    except ZeroDivisionError:
        return "Math Error: Cannot divide by zero!"
```